

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Jamal Tannous 208912337

Dan Ydov 039992391

Basel Mansour 208204859

Source Code Repository: [Link](#)

Video Demo: [Link](#)

Chapter 1: Introduction & Executive Summary	6
1.1 Project Vision	6
1.2 Problem Domain	6
1.3 Engineering Objective	6
1.4 Final Implemented System	7
1.5 Key Achievements	7
1.6 Executive Summary of Results	8
1.7 Report Roadmap	9
Chapter 2: Final Requirements & Design Recap	9
2.1 Final Problem Definition	9
2.2 Environment Taxonomy	9
2.3 Final Agent Roles	10
2.4 Final Agent Architecture	11
2.4.1 Beliefs	11
2.4.2 Desires / Goals	11
2.4.3 Intentions / Plans	11
2.5 Final Functional Requirements	11
2.6 Final Interaction Model	12
2.7 Final Task Allocation Design	13
2.8 Final Path Planning Design	13
2.9 Final Conflict Resolution Design	14
2.10 Final Deadlock Detection and Recovery Design	14
2.11 Final Simulation and Visualization Requirements	14
2.12 Justified Design Changes from the Original SDD	15
2.13 Final Requirements Traceability Summary	16
Chapter 3: Technical Implementation	17
3.1 Implementation Overview	17
3.2 Development Stack	17
3.3 Source Code Structure	18
3.4 Main Software Architecture	19
3.5 Simulation Engine Mechanics	20
3.6 Scenario Configuration	21
3.7 Task Lifecycle Implementation	21

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.8 Agent State Implementation.....	22
3.9 Advanced Task Allocation Implementation.....	23
3.10 Naive Baseline Implementation.....	23
3.11 Path Planning Implementation.....	24
3.11.1 Standard A*.....	24
3.11.2 Cooperative A*.....	24
3.11.3 Wait Action.....	24
3.12 Reservation Table Implementation.....	24
3.13 Conflict Detection Implementation.....	25
3.14 Priority-Based Conflict Resolution Implementation.....	25
3.15 Deadlock Detection and Recovery Implementation.....	26
3.16 Metrics and Logging Implementation.....	26
3.16.1 System-Level Metrics.....	26
3.16.2 Agent-Level Metrics.....	27
3.16.3 Exported Files.....	27
3.17 Utility and Social Welfare Implementation.....	28
3.18 Visualization Implementation.....	28
3.19 Reset and Reproducibility.....	29
3.20 Implementation Challenges and Fixes.....	30
3.20.1 Reset Behavior.....	30
3.20.2 Deadlock Overcounting.....	30
3.20.3 Agent Time Accounting.....	30
3.20.4 GUI Observability.....	30
3.20.5 Baseline Comparison.....	30
3.21 Technical Summary.....	30
Chapter 4: Agent Logic & Action Strategies.....	31
4.1 Overview.....	31
4.2 Agent Decision Model.....	31
4.2.1 Beliefs.....	31
4.2.2 Desires / Goals.....	32
4.2.3 Intentions / Current Plan.....	32
4.3 Agent State Machine.....	33
4.5 Naive Baseline Strategy.....	35
4.6 Advanced Auction-Based Task Allocation.....	36
4.7 Why Auction Allocation Is Better Than Nearest-Agent Assignment.....	37
4.8 Cooperative A* Strategy.....	38
4.9 Reservation Table Strategy.....	39
4.10 Movement Intention and Conflict Detection.....	41
4.11 Priority-Based Conflict Resolution.....	42

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.12 Dynamic Replanning Strategy.....	43
4.13 Deadlock Detection Strategy.....	44
4.14 Deadlock Recovery Strategy.....	45
4.15 Congestion-Aware Strategy.....	46
4.16 Pickup and Delivery Strategy.....	47
4.17 Utility and Social Welfare Strategy.....	48
4.18 Strategy Selection in the Simulator.....	49
4.19 Full Advanced Strategy Flow.....	50
4.20 Why the Advanced Strategy Is Suitable.....	52
4.21 Chapter Summary.....	53
Chapter 5: Simulator & Visualization Platform.....	53
5.1 Overview.....	53
5.2 Simulator Purpose.....	53
5.3 Simulator Execution Model.....	54
5.4 Running the Simulator.....	54
5.5 Installation Requirements.....	55
5.6 Supported Scenarios.....	55
5.7 Strategy Selection.....	55
5.8 Dashboard Layout.....	56
5.9 Grid Visualization.....	57
5.10 Agent Visualization.....	57
5.11 Task Visualization.....	58
5.12 Path and Reservation Visualization.....	59
5.13 Congestion Visualization.....	59
5.14 Conflict and Invalid Move Visualization.....	59
5.15 Deadlock Visualization.....	60
5.16 GUI Controls.....	60
5.17 Reset Behavior.....	60
5.18 Live Metrics Panel.....	61
5.19 Event Log.....	61
5.20 Exported Output Files.....	62
5.22 Observer Verification Checklist.....	62
5.23 Relationship Between GUI and Validation Results.....	63
5.24 Limitations of the Simulator.....	64
5.25 Chapter Summary.....	64
Chapter 6: Results & Validation Analysis.....	65
6.1 Overview.....	65
6.2 Validation Objectives.....	65
6.3 Evaluation Environment.....	65

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

6.4 Evaluation Metrics.....	66
6.4.1 System-Level Metrics.....	66
6.5 Strategy Comparison: Naive Baseline vs. Advanced Strategy.....	68
6.6 Advanced Strategy Scenario Results.....	70
6.7 Throughput Analysis.....	71
6.8 Average Completion Time Analysis.....	72
6.9 Social Welfare Analysis.....	73
6.10 Average Utility per Agent Analysis.....	74
6.11 Invalid Move Prevention Analysis.....	75
6.12 Deadlock Analysis.....	76
6.13 Stress Testing.....	77
6.13.1 High-Load Scenario.....	77
6.13.2 High-Density Scenario.....	78
6.13.4 Long High-Load Run: 6000 Timesteps.....	79
6.14 Requirement Validation.....	80
6.15 Main Findings.....	81
6.15.1 The Advanced Strategy Outperforms the Naive Baseline.....	81
6.15.2 Collision Avoidance Works.....	81
6.15.3 Higher Throughput Does Not Always Mean Better System Health.....	81
6.15.4 More Agents Help Only Until Congestion Costs Dominate.....	81
6.15.5 Deadlocks Are Manageable, Not Impossible.....	81
6.16 Threats to Validity.....	82
6.17 Final Validation Conclusion.....	82
Chapter 7: Social, Ethical & Future Work.....	83
7.1 Overview.....	83
7.2 Social Impact.....	83
7.3 Ethical Considerations.....	83
7.3.1 Safety.....	83
7.3.2 Transparency and Explainability.....	84
7.3.3 Accountability.....	84
7.3.4 Fairness Between Agents.....	84
7.3.5 Productivity vs. Safety.....	84
7.4 Limitations.....	85
7.5 Future Work.....	85
7.5.1 Multi-Seed Statistical Evaluation.....	85
7.5.2 Baseline Comparison Across All Scenarios.....	85
7.5.3 Energy-Aware Coordination.....	86
7.5.4 More Realistic Warehouse Layouts.....	86
7.5.5 Dynamic Obstacles and Human Workers.....	86

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

7.5.6 Communication Failure Modeling.....	86
7.5.7 Learning-Based Adaptation.....	86
7.5.8 Replay and Debugging Tools.....	87
7.7 Final Project Conclusion.....	87
References.....	89
Appendix A: How to Run the Simulator.....	89
A.1 Requirements.....	89
A.3 Running the Naive Baseline.....	90
A.4 Running the Strategy Comparison.....	90
A.5 Running All Validation Scenarios.....	90
A.6 Running the Long Stress Test.....	90
A.7 GUI Controls.....	90
Appendix B: Scenario Configuration.....	91
Appendix C: Validation Evidence.....	91
C.1 Strategy Comparison Summary.....	91
C.2 Advanced Scenario Summary.....	92
C.3 Long High-Load Stress Test.....	92

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Chapter 1: Introduction & Executive Summary

1.1 Project Vision

Swarm Logistics & Warehouse Automation is a multi-agent system project that simulates autonomous warehouse robots operating in a shared grid-based environment. The project addresses a realistic coordination problem: multiple agents must complete pickup-and-delivery tasks while sharing limited space, avoiding collisions, handling congestion, and resolving deadlocks.

The system is designed as a decentralized warehouse automation simulation. Instead of using a single central controller to assign tasks and control movement, each robot agent makes decisions based on its own state, the available task pool, communication with other agents, and shared coordination structures such as movement intentions and reservations.

The project vision is to demonstrate how coordinated local decision-making can produce safe and efficient global warehouse behavior. This is a core Multi-Agent Systems problem because the agents are autonomous, operate concurrently, interact within a shared environment, and must balance individual task execution with system-wide performance.

The OnePager defined the project as a decentralized MAS for warehouse automation, where autonomous robots collaborate to fulfill orders efficiently under dynamic conditions. It also identified the core strategies: congestion-aware auction-based task allocation, Cooperative A* path planning, fallback A*, wait-for graph deadlock detection, local intent communication, and dynamic replanning.

1.2 Problem Domain

Warehouse automation is a natural domain for Multi-Agent Systems because the environment contains many independent robots, shared physical resources, dynamic task arrivals, and potential movement conflicts.

The main problem is not simply finding a path for one robot. The real challenge is coordinating many agents that may compete for the same corridors, pickup stations, drop-off locations, and grid cells. If coordination is weak, agents may block each other, collide, repeatedly replan, or create deadlocks.

The project models this problem as a discrete warehouse grid. Each robot can move one cell per timestep, pick up items, deliver items, wait, bid for tasks, communicate short-term movement intentions, and replan when blocked. Tasks are generated dynamically during runtime, so agents cannot know all future work in advance.

The SRS defines the environment as multi-agent, collaborative, partially observable, dynamic, discrete, deterministic in movement, and stochastic in task arrivals. These properties make the problem suitable for evaluating decentralized coordination strategies.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

1.3 Engineering Objective

The main engineering objective is to design, implement, and validate a multi-agent warehouse simulator that can:

1. Generate pickup-and-drop-off tasks dynamically.
2. Assign tasks to agents using a decentralized-style auction mechanism.
3. Plan safe paths for multiple agents in a shared grid.
4. Prevent vertex conflicts and edge-swap conflicts.
5. Detect and recover from deadlocks.
6. Handle congestion through bidding and replanning.
7. Visualize agent behavior in real time.
8. Export metrics and logs for validation.

The system is considered successful if it achieves high task throughput, low average task completion time, zero collision rate, stable agent utilization, bounded congestion, and finite deadlock recovery time.

The SRS explicitly defines the target success criteria as high throughput, low average completion time, zero collision rate, high utilization, efficient path usage, low congestion, and fast deadlock resolution.

1.4 Final Implemented System

The final implemented system is a Python-based multi-agent warehouse simulation. It includes both an advanced coordination strategy and a naive baseline strategy for comparison.

The advanced strategy includes:

1. Deterministic auction-based task allocation.
2. Congestion-aware bid calculation.
3. Cooperative A* path planning.
4. Time-space reservation table.
5. Vertex and edge-swap conflict prevention.
6. Priority-based conflict resolution.
7. Dynamic replanning.
8. Wait-for graph deadlock detection.
9. Deadlock recovery through reservation release and replanning.
10. Real-time Pygame visualization.
11. Structured JSON/CSV logging.
12. Scenario-based validation and stress testing.

The naive baseline strategy includes:

1. Nearest-agent task assignment.
2. Standard A* path planning.
3. Current occupancy checking.
4. Waiting when blocked.
5. No auction bidding.
6. No Cooperative A*.
7. No future reservation table.
8. No edge-swap prevention.
9. No explicit deadlock recovery.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The naive baseline is included to prove that the advanced coordination mechanisms are not decorative. They materially improve safety, task completion, invalid-move prevention, utilization, utility, and social welfare.

1.5 Key Achievements

The final implementation achieved the following:

1. A complete runnable simulation engine was implemented.
2. Multiple warehouse agents can operate concurrently in the same grid.
3. Dynamic tasks are generated and tracked through the full lifecycle: unassigned, assigned, and completed.
4. Agents use deterministic bidding to assign tasks consistently.
5. Cooperative A* and reservations prevent future cell and edge conflicts.
6. Deadlock detection and recovery are implemented and validated.
7. A real-time Pygame dashboard visualizes agents, tasks, paths, reservations, congestion, conflicts, and metrics.
8. Validation scenarios were executed for baseline, high load, high density, deadlock, dynamic environment, and scalability conditions.
9. A direct naive-vs-advanced strategy comparison was implemented.
10. The advanced strategy achieved zero actual collisions across the tested advanced scenarios.
11. Utility and social welfare metrics were computed and visualized.
12. Logs and metrics were exported for reproducible validation.

The validation report shows that the advanced strategy completed 42 tasks in the baseline scenario compared with 12 tasks for the naive baseline, while also eliminating collisions and improving social welfare from -57.80 to 387.35.

1.6 Executive Summary of Results

The final system was validated using several scenario types. The main advanced-strategy results are summarized below.

Scenario	Agents	Tasks Created	Tasks Completed	Throughput	Avg Completion Time	Collisions	Deadlocks	Social Welfare
baseline	6	58	42	0.140	44.10	0	4	387.35
high_load	8	128	49	0.163	85.86	0	16	389.80
high_density	14	73	57	0.190	47.61	0	16	273.75
deadlock	4	99	42	0.140	90.69	0	8	382.90
dynamic	8	81	56	0.187	59.12	0	9	506.70
scalability	12	96	77	0.257	41.10	0	22	620.60

Table 1: Summary of advanced-strategy performance across the main validation scenarios.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The most important result is safety: the advanced strategy produced zero actual collision events in all tested scenarios. This directly supports the SRS collision avoidance requirement, which states that no two agents may occupy the same grid cell at the same timestep.

The scalability scenario produced the highest throughput and social welfare, showing that adding agents can improve system-level productivity when the warehouse is not overly constrained. The high-density scenario, however, showed that adding too many agents increases congestion and reduces average utility per agent. This confirms a key MAS tradeoff: more agents improve parallelism only until coordination costs begin to dominate.

1.7 Report Roadmap

This report documents the full engineering lifecycle of the project.

Chapter 1 introduces the project vision, domain, engineering objective, implemented system, and key results.

Chapter 2 summarizes the final requirements and design, including the environment taxonomy, agent roles, functional requirements, and justified design changes.

Chapter 3 explains the technical implementation, software architecture, development stack, module responsibilities, and simulation engine mechanics.

Chapter 4 presents the agent logic and action strategies, including auction bidding, Cooperative A*, conflict prevention, deadlock detection, replanning, naive baseline behavior, and pseudo-code.

Chapter 5 explains the simulator and visualization platform, including the user manual, GUI dashboard, controls, visualization elements, and logging outputs.

Chapter 6 presents the results and validation analysis, including baseline-vs-advanced comparison, scenario testing, utility, social welfare, stress tests, and discussion.

Chapter 7 discusses social and ethical considerations, limitations, and future work.

Chapter 2: Final Requirements & Design Recap

2.1 Final Problem Definition

The final project implements a decentralized-style multi-agent warehouse automation simulator. The warehouse contains autonomous robot agents that must complete dynamically generated pickup-and-delivery tasks while operating in a shared grid environment.

Each task has a pickup location and a drop-off location. An agent assigned to a task must navigate to the pickup location, collect the item, then navigate to the drop-off location and complete the delivery. Multiple agents operate simultaneously, so the system must prevent invalid movement, collisions, edge swaps, unresolved blocking, and deadlocks.

The SRS defines the project scope as simulation-based. Physical deployment, real robot hardware, network failures, and real warehouse integration are outside the scope.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

2.2 Environment Taxonomy

The final environment follows the taxonomy defined in the SRS and OnePager.

Property	Final Interpretation
Multi-agent	Multiple autonomous warehouse robots operate concurrently.
Collaborative	Agents share the global objective of maximizing warehouse throughput.
Partially observable	Agents know the static map and their own state, but future task arrivals are unknown.
Dynamic	New tasks are generated during runtime.
Discrete	Space is represented as grid cells and time advances in discrete timesteps.
Deterministic actions	Movement results are predictable if the action is valid.
Stochastic task arrivals	Tasks appear dynamically based on scenario task-generation rates.
Synchronous	All agents choose and execute actions once per timestep.
Shared environment	Agents compete for grid cells, corridors, pickup zones, and drop-off zones.

Table 2: Classification of the warehouse simulation environment according to MAS properties.

This environment is appropriate for MAS analysis because system behavior emerges from the interaction between multiple autonomous agents rather than from a single decision-maker.

2.3 Final Agent Roles

The final system uses homogeneous robot agents. All agents have the same capabilities and decision logic. There are no specialized robot types.

Each agent can perform the following roles depending on its current state:

1. Idle agent: waits for available tasks and avoids blocking active agents.
2. Bidding agent: evaluates available tasks and submits bid values.
3. Assigned agent: owns one task and plans toward the pickup location.
4. Pickup agent: reaches the pickup cell and collects the item.
5. Delivery agent: carries the item toward the drop-off location.
6. Waiting agent: waits because of a conflict, reservation, blockage, or priority decision.
7. Replanning agent: releases old reservations and computes a new path.
8. Deadlock recovery participant: participates in resolving cyclic waiting.

The SDD defines each robot as a hybrid deliberative-reactive agent. The deliberative part handles task selection, bidding, planning, and goal-directed behavior; the reactive part handles immediate safety, conflicts, blockage, and replanning.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

2.4 Final Agent Architecture

The final agent architecture follows a simplified BDI-inspired model combined with reactive safety rules.

2.4.1 Beliefs

Each agent maintains beliefs about:

1. Its current position.
2. The static warehouse layout.
3. Obstacles, pickup cells, and drop-off cells.
4. Its assigned task, if any.
5. Available tasks.
6. Current path.
7. Nearby agents or occupied cells.
8. Reservations.
9. Congestion.
10. Waiting and blocking status.

2.4.2 Desires / Goals

Each agent aims to:

1. Complete assigned tasks.
2. Minimize task completion time.
3. Avoid collisions.
4. Avoid edge swaps.
5. Avoid contributing to congestion.
6. Recover from deadlocks.
7. Support global throughput.
8. Stay productive when tasks are available.

2.4.3 Intentions / Plans

Each agent may commit to:

1. Bidding for a task.
2. Moving toward pickup.
3. Picking up an item.
4. Moving toward drop-off.
5. Delivering an item.
6. Waiting to avoid a conflict.
7. Replanning because of blockage.
8. Participating in deadlock recovery.

The final state machine follows the SDD states: IDLE, BIDDING, ASSIGNED, MOVING_TO_PICKUP, PICKING_UP, MOVING_TO_DROPOFF, DELIVERING, WAITING, REPLANNING, and DEADLOCK_RECOVERY.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

2.5 Final Functional Requirements

Requirement Area	Final Implementation
Task generation	Tasks are generated dynamically during runtime.
Task pool management	Tasks are stored as unassigned, assigned, or completed.
Task assignment consistency	Deterministic auction ensures each task is assigned to at most one agent.
Task lifecycle	Tasks move from unassigned to assigned to completed.
Collision avoidance	Vertex conflicts are prevented using reservations and movement validation.
Movement validity	Agents cannot move outside the grid, into obstacles, or into invalid cells.
Deadlock detection and recovery	Wait-for graph and persistent blocking logic detect and recover from deadlocks.
Congestion handling	Congestion affects bidding and path planning.
Task completion tracking	Completion counts and completion times are logged.
System continuity	Simulation continues while tasks are available or generated.
Agent path planning	Agents use A* and Cooperative A* depending on strategy.
Dynamic replanning	Agents replan when blocked, delayed, or involved in conflict.
Communication usage	Message-style coordination is modeled through bids, movement intentions, reservations, and logs.
Visualization	GUI shows agents, tasks, paths, reservations, congestion, and event information.
Logging	JSON/CSV logs record metrics, events, decisions, and snapshots.

Table 3: Functional requirements implemented in the final MAS simulator.

The SRS requires collision avoidance, movement validity, deadlock recovery, congestion handling, decentralized task allocation, communication usage, and task completion confirmation.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

2.6 Final Interaction Model

The final interaction model is decentralized in logic but executed inside a synchronous simulation loop for reproducibility.

Agents coordinate through:

1. Task auctions.
2. Bid submission.
3. Deterministic winner selection.
4. Movement intention exchange.
5. Reservation-table coordination.
6. Priority-based conflict handling.
7. Deadlock detection and recovery.
8. Event and decision logging.

The SDD uses a FIPA-ACL-inspired communication model, with message purposes such as CFP, PROPOSE, ACCEPT_PROPOSAL, REJECT_PROPOSAL, INFORM, and REQUEST. In the final implementation, this is represented through structured auction events, movement-intention logic, reservations, and event logs rather than real network messaging.

This is an intentional simulation design choice. The simulator is not a physical distributed robot platform. It is a controlled MAS simulation where decentralized-style decisions are implemented inside a deterministic execution engine.

2.7 Final Task Allocation Design

Task allocation is performed using deterministic auction logic.

For each available task:

1. Idle agents evaluate the task.
2. Each candidate agent computes a bid cost.
3. The bid includes travel distance, congestion, workload, and reservation-delay effects.
4. The lowest bid wins.
5. If two bids are equal, the lowest agent ID wins.
6. The task becomes assigned to the winning agent.
7. The winner plans a path to the pickup location.

The final bid-cost logic follows the SDD design:

$$\text{BidCost} = \alpha \times \text{Distance} + \beta \times \text{Congestion} + \gamma \times \text{Workload} + \delta \times \text{ReservationDelay}$$

The SDD defines this bid model and explains that the lowest bid represents the agent expected to complete the task with the lowest coordination cost.

2.8 Final Path Planning Design

The advanced strategy uses Cooperative A* with a reservation table.

Standard A* searches over spatial cells only. Cooperative A* searches over:

State = (x, y, t)

This means each path decision considers both location and timestep. This is necessary because two agents may be able to use the same cell safely at different times, but not at the same time.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The planner rejects moves that:

1. Leave the grid.
2. Enter obstacles.
3. Enter a reserved cell at the same future timestep.
4. Create an edge-swap conflict.
5. Violate occupancy constraints.
6. Cause invalid movement.

The SDD defines Cooperative A* as the primary planner and explicitly states that it searches in space and time using cell plus timestep states.

2.9 Final Conflict Resolution Design

The final system detects and prevents:

1. Vertex conflicts: two agents attempt to enter the same cell at the same timestep.
2. Edge conflicts: two agents attempt to swap positions in the same timestep.
3. Blocking conflicts: an agent tries to enter a cell occupied by a waiting or blocked agent.
4. Reservation conflicts: an agent attempts to use a future cell or edge already reserved by another agent.

When a conflict is detected, priority rules determine which agent may proceed. Priority considers whether the agent is carrying an item, waiting time, progress toward task completion, and deterministic agent ID tiebreaking.

The SDD defines these conflict types and priority rules, including carrying item, waiting time, progress, and lower agent ID as deterministic tiebreaker.

2.10 Final Deadlock Detection and Recovery Design

The final system detects deadlocks using waiting relationships. A deadlock exists when agents wait for each other in a cycle.

Example:

A1 waits for A2

A2 waits for A3

A3 waits for A1

This creates the cycle:

A1 → A2 → A3 → A1

When a persistent deadlock is detected:

1. The involved agents are identified.
2. The system selects the highest-priority agent.
3. Lower-priority agents release future reservations.
4. Lower-priority agents replan paths.
5. Agents resume movement after the cycle is broken.

The SDD specifies wait-for graph deadlock detection and deterministic recovery through reservation release and replanning.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

2.11 Final Simulation and Visualization Requirements

The final simulator satisfies the SRS visualization and observability requirements through a real-time Pygame dashboard and structured logs.

The simulator supports:

1. Grid visualization.
2. Obstacles.
3. Pickup and drop-off stations.
4. Agent IDs and states.
5. Planned paths.
6. Reservations.
7. Congestion heatmap.
8. Task links.
9. Conflict indicators.
10. Deadlock indicators.
11. Live metrics.
12. Event feed.
13. Pause, step, reset, and toggle controls.
14. JSON and CSV exports.

The SRS requires real-time graphical visualization of the grid, obstacles, pickup/drop-off locations, agent positions, task assignments, planned/reserved paths, and congestion areas.

2.12 Justified Design Changes from the Original SDD

The final implementation follows the SDD closely, but several design changes were made during implementation and validation.

Design Change	Original Direction	Final Implementation	Justification
MessageBus abstraction simplified	SDD described explicit FIPA-style messages	Final implementation models communication through structured events, auctions, reservations, and logs	Full network-style messaging was unnecessary for a deterministic simulation; the coordination semantics are preserved.
Centralized simulation loop clarified	SDD emphasized decentralized agents	Final simulator uses a centralized synchronous loop but decentralized-style decision logic	A simulation engine must advance global time and enforce consistency; it is not a central controller assigning all decisions.
Naive baseline added	SDD focused on advanced coordination	Final implementation includes naive nearest-agent baseline	Needed for stronger validation and required baseline-vs-advanced comparison.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Deadlock detection made persistent	SDD described wait-for graph cycles	Final implementation avoids overcounting transient one-step waits	Prevents false deadlock inflation and improves metric validity.
GUI upgraded	SDD described dashboard concept	Final GUI uses improved robot shapes, pickup/drop-off icons, task links, reservations, heatmap, and side panel	Makes validation and demonstration clearer.
Utility/social welfare derived from logs	SRS/validation expected utility metrics	Final report computes utility/social welfare from exported metrics	Allows rigorous comparison without hard-coding utility as the only internal optimization target.
Scenario suite finalized	SRS described validation scenarios	Final implementation includes baseline, high_load, high_density, deadlock, dynamic, and scalability	Matches SRS validation plan and supports stress testing.

Table 4: Justified Design Changes from the Original SDD

These changes are engineering refinements, not deviations from the project goals. They improve reproducibility, validation strength, observability, and grading clarity.

2.13 Final Requirements Traceability Summary

Requirement Category	Evidence in Final System
Dynamic tasks	Task generation and task pool logs.
Decentralized task allocation	Auction protocol and deterministic bid winner selection.
Single task ownership	Task assignment state ensures one owner per task.
Path planning	A* and Cooperative A* planners.
Collision avoidance	Reservation table, conflict resolver, final safety gate.
Edge-swap prevention	Edge reservations and swap checks.
Deadlock recovery	Wait-for graph and replanning.
Congestion handling	Congestion-aware bid/path costs and heatmap.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Visualization	Pygame dashboard.
Logging	JSON/CSV summaries, events, decisions, snapshots.
Validation	Scenario results, utility, social welfare, naive-vs-advanced comparison.

Table 5: Traceability between project requirements and final validation evidence.

Chapter 3: Technical Implementation

3.1 Implementation Overview

The final system was implemented as a Python-based discrete-time multi-agent warehouse simulator. The simulator models a shared warehouse grid, autonomous robot agents, dynamic pickup-and-delivery tasks, task allocation, path planning, collision prevention, deadlock detection, metrics collection, and real-time visualization.

The implementation follows the SDD architecture closely. The SDD specified a modular Python simulator with major components such as WarehouseSimulation, WarehouseGrid, RobotAgent, TaskPool, AuctionProtocol, PathPlanner, ReservationTable, ConflictResolver, DeadlockDetector, MetricsCollector, and VisualizationDashboard.

The final implementation also adds a naive baseline strategy so that the advanced agent strategy can be compared against a simpler greedy controller. This addition improves the scientific validity of the final validation because it demonstrates that the advanced coordination mechanisms produce measurable improvement over a weaker baseline.

The implementation is not a physical robot controller. It is a deterministic simulation platform designed to evaluate multi-agent coordination strategies under controlled and reproducible conditions.

3.2 Development Stack

The selected development stack is intentionally simple, deterministic, and suitable for fast experimentation.

Component	Technology / Method	Purpose
Programming language	Python 3.x	Main implementation language for simulation, agents, analytics, and scenario execution.
Simulation model	Custom synchronous grid simulator	Provides full control over discrete timesteps, action ordering, conflict checking, and reproducibility.
Path planning	Custom A* and Cooperative A*	Computes paths while respecting grid constraints and time-space reservations.
Data modeling	Python dataclasses, enums, dictionaries, and lists	Represents agents, tasks, reservations, logs, metrics, and scenario configuration.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Visualization	Pygame	Real-time dashboard for observing agents, tasks, paths, reservations, congestion, and events.
Logging	JSON and CSV	Exports summaries, events, snapshots, and agent decision traces.
Experiment control	Python scripts	Runs individual scenarios, all scenarios, and naive-vs-advanced comparison.

Table 6: Development stack used for the MAS simulation platform.

This stack matches the SDD design, which selected Python 3.x, a custom synchronous grid-based simulator, custom A*/Cooperative A*, dataclasses/enums/dictionaries/lists, Pygame visualization, and structured CSV/JSON analytics.

The reason for using a custom simulator instead of a generic agent-based modeling framework is control. The project requires strict synchronized movement, deterministic tie-breaking, cell reservations, edge-swap prevention, and exact validation logs. A custom simulator makes these behaviors explicit and inspectable.

3.3 Source Code Structure

The final project is organized as a modular Python package.

Project structure:

```
swarm_warehouse/  
  agent.py  
  auction.py  
  conflict.py  
  deadlock.py  
  grid.py  
  metrics.py  
  models.py  
  path_planner.py  
  reservation.py  
  simulation.py  
  task_pool.py  
  visualization.py  
  
run_simulation.py  
run_all_scenarios.py  
run_strategy_comparison.py  
requirements.txt  
README.md
```

Fig 1: Overview of the final source-code structure.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Main files:

File	Responsibility
models.py	Defines shared data models such as agent state, task state, task objects, positions, events, and scenario configuration.
grid.py	Implements the warehouse grid, obstacles, pickup/drop-off cells, valid movement, and occupancy checks.
task_pool.py	Manages task creation, assignment, completion, and task lifecycle state.
agent.py	Implements robot agent state, task ownership, path following, pickup/drop-off behavior, waiting, and replanning counters.
auction.py	Implements task bidding, deterministic winner selection, and naive nearest-agent assignment support.
path_planner.py	Implements standard A* and Cooperative A* path planning.
reservation.py	Implements vertex and edge reservations for future timesteps.
conflict.py	Detects and prevents vertex conflicts, edge conflicts, blocking conflicts, and invalid moves.
deadlock.py	Detects persistent blocking and wait-for cycles, then triggers recovery behavior.
metrics.py	Collects throughput, completion time, collisions, invalid moves, deadlocks, utilization, utility, and social welfare.
visualization.py	Implements the Pygame dashboard, rendering, controls, heatmap, paths, reservations, agents, tasks, and event feed.
simulation.py	Coordinates the full simulation loop, scenario configuration, reset behavior, strategy mode, logging, and output export.
run_simulation.py	CLI entry point for running a single scenario.
run_all_scenarios.py	Runs the full advanced-strategy validation scenario suite.
run_strategy_comparison.py	Runs the naive-vs-advanced baseline comparison.

Table 6: Overview of the final source-code main files.

This structure directly realizes the SDD software architecture. The SDD describes the same major components and responsibilities: simulation control, warehouse grid, robot agents, task

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

pool, auction protocol, path planning, reservations, conflict resolution, deadlock detection, metrics, and visualization.

3.4 Main Software Architecture

The implementation follows a layered architecture:

1. Environment layer
Contains the warehouse grid, obstacles, pickup locations, drop-off locations, occupancy checks, and valid-neighbor logic.
2. Agent layer
Contains robot agents, their current state, assigned task, path, waiting time, replanning count, and action execution.
3. Coordination layer
Contains auction allocation, reservation table, conflict detection, priority handling, and deadlock recovery.
4. Simulation layer
Advances global time, generates tasks, calls coordination logic, executes movements synchronously, records logs, and exports results.
5. Visualization and analytics layer
Displays the live simulation and exports JSON/CSV files for validation.

The simulation engine advances time, but it does not act as an intelligent central planner. The advanced strategy still models decentralized-style agent decisions: agents bid for tasks, follow their own paths, wait or replan based on conflicts, and participate in deadlock recovery. The centralized loop exists only to enforce synchronous simulation semantics and deterministic validation.

This distinction is important. The SRS describes the system as decentralized in coordination, but the implementation is a simulation platform. In simulation, a global loop is necessary to apply actions consistently, detect conflicts, update the grid, and collect metrics.

3.5 Simulation Engine Mechanics

The main simulation loop is implemented in WarehouseSimulation.

At each timestep, the simulator performs the following sequence:

1. Generate new tasks according to the scenario task-generation rate.
2. Add new tasks to the task pool.
3. Run task allocation according to the selected strategy.
4. For advanced strategy, collect bids and assign tasks through deterministic auction logic.
5. For naive strategy, assign tasks to nearest idle agents.
6. For each assigned agent, ensure that a path exists toward the current goal.
7. For advanced strategy, use Cooperative A* and reservations.
8. For naive strategy, use standard A* and current occupancy checks.
9. Collect intended movements from agents.
10. Detect invalid moves, vertex conflicts, edge conflicts, and blocking conflicts.
11. Apply priority and safety rules.
12. Detect persistent blocking and deadlocks.
13. Resolve deadlocks by releasing reservations and replanning.
14. Execute approved actions synchronously.
15. Update task states.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

16. Update agent states and grid occupancy.
17. Record event logs, agent decisions, snapshots, and metrics.
18. Render the dashboard if visualization is enabled.
19. Continue until the maximum timestep is reached.

This corresponds closely to the SDD simulation loop, which requires task generation, agent perception, task allocation, path planning, reservation checks, conflict resolution, deadlock detection, action execution, metric recording, visualization, and export.

3.6 Scenario Configuration

The implementation supports multiple predefined scenarios. Each scenario changes the number of agents, task-generation pressure, warehouse layout, and stress conditions.

Scenario	Purpose
baseline	Normal operating condition.
high_load	Tests behavior when tasks are generated frequently.
high_density	Tests congestion when many agents share the same grid.
deadlock	Uses a narrow-corridor layout to provoke blocking and deadlocks.
dynamic	Tests behavior under medium-load dynamic task generation.
scalability	Tests performance as the number of agents increases.
long high-load	Tests extended stability over 6000 timesteps.

Table 7: Supported scenario configurations used by the simulator.

The SRS already defines the required validation scenarios: baseline, high load, high density, deadlock, dynamic environment, and scalability. The implementation directly follows this scenario plan.

The simulator also supports command-line control for the scenario, strategy, number of timesteps, rendering, and output directory.

Example commands:

- `python run_simulation.py --scenario baseline --strategy advanced --steps 300 --out runs/baseline_advanced`
- `python run_simulation.py --scenario baseline --strategy naive --steps 300 --out runs/baseline_naive`
- `python run_simulation.py --scenario high_load --strategy advanced --steps 6000 --out runs/high_load_6000`
- `python run_all_scenarios.py`
- `python run_strategy_comparison.py`

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.7 Task Lifecycle Implementation

Tasks are represented as pickup-and-drop-off jobs.

Each task contains:

1. Task ID.
2. Pickup cell.
3. Drop-off cell.
4. Task state.
5. Assigned agent ID.
6. Creation time.
7. Completion time.

The implemented task lifecycle is:

UNASSIGNED → ASSIGNED → COMPLETED

A task begins as unassigned when generated. It becomes assigned when a winning agent is selected by the auction or naive assignment logic. It becomes completed when the assigned agent reaches the drop-off cell after pickup.

This directly satisfies the SRS requirement that every task must exist as unassigned, assigned, or completed, and that tasks transition from unassigned to assigned and then to completed.

The TaskPool module manages all active tasks. It provides:

1. Task creation.
2. Available task retrieval.
3. Assignment update.
4. Completion update.
5. Active/pending/completed task counts.
6. Data used by metrics and visualization.

3.8 Agent State Implementation

Each robot agent is implemented as a finite-state autonomous entity.

The main agent states are:

State	Meaning
IDLE	Agent has no assigned task.
ASSIGNED	Agent owns a task but may not yet be moving.
MOVING_TO_PICKUP	Agent is moving toward the pickup cell.
PICKING_UP	Agent is collecting the item.
MOVING_TO_DROPOFF	Agent is carrying the item toward the destination.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

DELIVERING	Agent is completing the drop-off.
WAITING	Agent is blocked or waiting because of conflict/reservation.
REPLANNING	Agent is computing a new path.
DEADLOCK_RECOVERY	Agent is involved in deadlock resolution.

Table 8: Agent states used in the final robot state machine.

The SDD defines this state-machine structure and transitions, including idle, bidding, assigned, moving to pickup, picking up, moving to drop-off, delivering, waiting, replanning, and deadlock recovery.

The implementation simplifies some purely communication-oriented states internally, but preserves the same lifecycle behavior. For example, bidding is represented by auction events and bid submissions rather than requiring the agent to remain visually in a long-lived BIDDING state.

3.9 Advanced Task Allocation Implementation

The advanced strategy uses deterministic auction-based task allocation.

For each unassigned task, idle agents compute a bid. The bid estimates how costly it would be for that agent to complete the task.

The bid considers:

1. Distance from the agent to the pickup cell.
2. Distance from pickup to drop-off.
3. Local or route congestion.
4. Workload / whether the agent is already busy.
5. Reservation delay or future path conflicts.

The lowest valid bid wins. If two bids are equal, the lower agent ID wins as a deterministic tiebreaker.

This implements the SDD auction model, where agents submit bids and the lowest bid wins, with deterministic tie-breaking to prevent inconsistent assignment.

The deterministic tiebreaker is important because the simulation must be reproducible. Without deterministic resolution, two agents might claim the same task or results might change between runs even with the same random seed.

3.10 Naive Baseline Implementation

The final implementation includes a naive baseline strategy to support direct comparison against the advanced strategy.

The naive baseline uses:

1. Nearest idle agent assignment.
2. Standard A* path planning.
3. Current occupancy checks.
4. Waiting when blocked.
5. No auction bidding.
6. No congestion-aware cost.
7. No Cooperative A*.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

8. No time-space reservation table.
9. No edge-swap prevention.
10. No explicit deadlock recovery.

This baseline is intentionally weaker. It represents a simple greedy approach that may appear reasonable in sparse conditions but breaks down under multi-agent congestion.

The baseline exists for validation, not because it is a recommended design. It allows the report to prove that the advanced strategy is not merely more complex, but measurably better.

The implemented comparison showed that the advanced strategy completed 42 tasks compared with 12 for the naive baseline, eliminated actual collisions, and improved social welfare from -57.80 to 387.35.

3.11 Path Planning Implementation

The path_planner.py module implements both standard A* and Cooperative A*.

3.11.1 Standard A*

Standard A* is used mainly by the naive baseline. It searches over grid cells and uses Manhattan distance as the heuristic.

The planner rejects:

1. Cells outside the grid.
2. Obstacles.
3. Invalid movement.
4. Currently occupied cells where applicable.

Standard A* is simple and fast, but it does not reason about future time. It cannot reliably prevent two agents from planning to occupy the same future cell or swapping positions through the same edge.

3.11.2 Cooperative A*

Cooperative A* is used by the advanced strategy. It searches over space-time states: state = (row, col, timestep)

This allows the planner to evaluate whether a cell is safe at a specific future timestep, not just whether it is physically reachable.

Cooperative A* rejects:

1. Obstacles.
2. Grid-boundary violations.
3. Reserved vertex conflicts.
4. Reserved edge conflicts.
5. Future cell conflicts.
6. Opposite-direction edge swaps.

The SDD defines Cooperative A* as the primary planner and explains that it searches over cell plus timestep states so agents can avoid future reserved cells and edge conflicts.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.11.3 Wait Action

The planner includes waiting as a valid action.

This is necessary because some valid multi-agent solutions require one agent to wait while another passes through a bottleneck. Without wait actions, narrow corridors would create unnecessary planning failures.

The SDD explicitly notes that wait must be included as a valid action because some narrow-corridor situations are solvable only if waiting is allowed.

3.12 Reservation Table Implementation

The reservation.py module implements future cell and edge reservations.

Two reservation types are used:

Reservation Type	Purpose
Vertex reservation	Prevents two agents from occupying the same cell at the same timestep.
Edge reservation	Prevents two agents from swapping positions through the same edge in opposite directions.

Table 9: Reservation types used for future cell and edge conflict prevention.

Each advanced agent reserves its planned path for a limited future horizon. When an agent replans or completes a task, old reservations are released.

The reservation table supports:

1. Reserving cells for future timesteps.
2. Reserving edge movements.
3. Checking whether a future cell is already reserved.
4. Checking whether an opposite-direction edge movement is reserved.
5. Releasing reservations for an agent.
6. Removing expired reservations.

This realizes the SDD design, which identifies the reservation table as essential for Cooperative A* and for preventing future collisions.

3.13 Conflict Detection Implementation

The conflict.py module handles immediate movement safety.

The simulator checks proposed movements before applying them. It detects:

1. Vertex conflicts: two agents want the same next cell.
2. Edge conflicts: two agents attempt to swap positions.
3. Blocking conflicts: an agent attempts to move into a cell occupied by a waiting or non-moving agent.
4. Reservation conflicts: the move violates a future reservation.
5. Invalid movement: move is outside the grid, into an obstacle, or otherwise illegal.

If a proposed move is unsafe, it is blocked before execution. This is why the validation report can distinguish between actual collision events and invalid moves prevented.

The SDD defines the same conflict types: vertex conflicts, edge conflicts, and blocking conflicts.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.14 Priority-Based Conflict Resolution Implementation

When multiple agents want incompatible movements, the conflict resolver applies deterministic priority rules.

Priority is based on:

1. Whether the agent is carrying an item.
2. Waiting time.
3. Progress toward completing the assigned task.
4. Agent ID as deterministic tiebreaker.

The higher-priority agent keeps its movement. Lower-priority agents wait or replan.

This matches the SDD priority design, which gives priority to agents carrying items, agents that waited longer, agents closer to task completion, and finally lower agent ID as a deterministic tiebreaker.

Deterministic priority is important because random conflict resolution would make the validation less reproducible.

3.15 Deadlock Detection and Recovery Implementation

The `deadlock.py` module detects persistent cyclic waiting or blocking.

The advanced implementation uses waiting relationships between agents. If an agent repeatedly waits because of another agent, a waiting edge is created. A set of waiting edges can form a cycle.

Example:

A1 waits for A2

A2 waits for A3

A3 waits for A1

This forms:

$A1 \rightarrow A2 \rightarrow A3 \rightarrow A1$

When a persistent cycle is detected:

1. Involved agents are identified.
2. The highest-priority agent is allowed to keep priority.
3. Lower-priority agents release future reservations.
4. Lower-priority agents enter replanning.
5. New paths are computed using updated reservations.
6. Movement resumes after the cycle is broken.

The SDD defines deadlock detection using a wait-for graph and recovery using priority, reservation release, and replanning.

A practical implementation adjustment was made during validation: deadlock detection requires persistence before declaring a real deadlock. This prevents ordinary one-step waiting from being incorrectly counted as a deadlock.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.16 Metrics and Logging Implementation

The metrics.py module records both system-level and agent-level data.

3.16.1 System-Level Metrics

System-level metrics include:

1. Total simulation time.
2. Tasks created.
3. Tasks completed.
4. Throughput.
5. Average completion time.
6. Collision events.
7. Collision rate.
8. Invalid moves prevented.
9. Deadlocks detected.
10. Average deadlock resolution time.
11. Agent utilization.
12. Social welfare.
13. Average utility.

The SRS defines these success metrics, including throughput, average task completion time, collision rate, utilization, path efficiency, deadlock resolution time, congestion index, agent utility, and social welfare.

3.16.2 Agent-Level Metrics

Agent-level metrics include:

1. Tasks completed by agent.
2. Active time.
3. Idle time.
4. Waiting time.
5. Current wait streak.
6. Replanning count.
7. Distance traveled.
8. Conflicts involved.
9. Deadlocks involved.
10. Average bid value.

These metrics are important because total throughput alone can hide unfair or inefficient agent behavior. For example, a system could have acceptable throughput while one agent does most of the work and others remain idle.

The SDD explicitly requires agent-level metrics such as tasks completed by agent, active time, idle time, waiting time, replanning count, distance traveled, conflicts involved, deadlocks involved, and average bid value.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.16.3 Exported Files

Each simulation run exports validation artifacts such as:

File	Purpose
summary.json	Final scenario metrics and agent metrics.
events.json / events.csv	Chronological task, movement, conflict, and deadlock events.
agent_decisions.json	Agent-level decision traces.
snapshots.json / snapshots.csv	Timestep-level system snapshots.
strategy_comparison_summary.json	Naive-vs-advanced comparison output.

Table 10: Exported simulation files and their purpose.

These outputs support reproducibility because the final report results can be traced back to generated data files.

3.17 Utility and Social Welfare Implementation

The final validation computes utility and social welfare from exported agent metrics.

For each agent i :

$$U_i = 10 \times \text{completed_tasks}_i \\ - 0.05 \times \text{waiting_time}_i \\ - 0.10 \times \text{replanning_count}_i \\ - 0.05 \times \text{conflicts_involved}_i \\ - 1.00 \times \text{deadlocks_involved}_i$$

Social welfare is:

$$\text{Social Welfare} = \sum U_i$$

Average utility is:

$$\text{Average Utility} = \text{Social Welfare} / \text{Number of Agents}$$

This utility definition rewards productive task completion and penalizes coordination costs. It is consistent with the SRS idea that system performance should maximize social welfare while minimizing total system cost and maintaining efficient coordination.

The utility function is derived from exported metrics rather than being the only internal decision objective. This is intentional. The agents use operational coordination mechanisms such as bidding, reservations, path planning, waiting, and replanning; utility is then used to evaluate the resulting behavior.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.18 Visualization Implementation

The visualization.py module implements the Pygame dashboard.

The dashboard displays:

1. Warehouse grid.
2. Obstacles.
3. Pickup stations.
4. Drop-off stations.
5. Robot agents with IDs.
6. Agent state indicators.
7. Direction arrows.
8. Package indicator when carrying an item.
9. Assigned and unassigned task links.
10. Planned paths.
11. Future reservations.
12. Congestion heatmap.
13. Conflict and invalid-move highlights.
14. Deadlock indicators.
15. Live metrics.
16. Agent state table.
17. Event feed.
18. Legend and GUI controls.

The SDD states that the visualization should show the warehouse grid, agents, tasks, planned paths, reserved cells, congestion, conflicts, deadlocks, state panel, event log, and observer validation information.

The implemented GUI also supports interaction controls:

1. Pause / resume.
2. Step once.
3. Reset.
4. Show/hide paths.
5. Show/hide reservations.
6. Show/hide congestion heatmap.
7. Show/hide task links.

This satisfies the SRS requirement for start, pause, reset, adjustable inspection, and step-by-step execution capabilities.

3.19 Reset and Reproducibility

The simulator includes a reset function that reconstructs the simulation state.

Reset clears and rebuilds:

1. Agents.
2. Grid occupancy.
3. Task pool.
4. Reservations.
5. Metrics.
6. Logs.
7. Random generator.
8. Timestep counter.
9. Scenario state.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

This is necessary because a visual reset that only moves the camera or clears the screen would not be a real simulation reset. The final reset behavior reproduces the selected scenario from timestep zero.

The simulator uses deterministic random seeds. Therefore, when the same scenario and seed are used, the same task-generation sequence and initial setup can be reproduced. This supports repeatable validation, which is required by the SRS. The SRS states that the simulator should support deterministic execution through configurable random seeds.

3.20 Implementation Challenges and Fixes

Several implementation issues were identified and corrected during development.

3.20.1 Reset Behavior

Initial issue: the GUI displayed reset controls, but the simulation state was not fully rebuilt.

Fix: a proper reset method was added to rebuild agents, grid state, task pool, reservations, metrics, logs, random generator, and timestep.

Result: reset now restarts the selected scenario correctly from timestep zero.

3.20.2 Deadlock Overcounting

Initial issue: transient one-step waits were sometimes counted as deadlocks.

Fix: deadlock detection was changed to require persistence before declaring a true deadlock.

Result: deadlock counts became realistic and defensible for the report.

3.20.3 Agent Time Accounting

Initial issue: some agent metrics had inconsistent time accounting, where active time, idle time, and waiting time did not sum correctly to the total simulation duration.

Fix: time accounting was corrected so each timestep contributes to exactly one relevant time category.

Result: agent metrics became valid for reporting and utility calculation.

3.20.4 GUI Observability

Initial issue: simple shapes made it difficult to understand what agents, tasks, pickup stations, and drop-off stations represented.

Fix: the GUI was improved with robot-like agent icons, pickup/drop-off station shapes, package indicators, direction arrows, task links, reservations, heatmap, conflict highlights, and a side panel.

Result: the dashboard became useful for demonstration and validation, not just animation.

3.20.5 Baseline Comparison

Initial issue: the project originally validated the advanced strategy across multiple scenarios but did not include a true numeric comparison against a simpler strategy.

Fix: a naive baseline strategy and `run_strategy_comparison.py` were added.

Result: the report can now show that the advanced strategy materially improves completion, safety, utility, and social welfare compared with a weaker baseline.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

3.21 Technical Summary

The final implementation is a complete modular simulation platform for decentralized-style warehouse coordination. It includes:

1. A deterministic grid simulation engine.
2. Autonomous robot agents.
3. Dynamic task generation.
4. Task lifecycle management.
5. Advanced auction-based coordination.
6. Naive baseline strategy.
7. Standard A* and Cooperative A*.
8. Time-space reservation table.
9. Vertex and edge-swap prevention.
10. Conflict resolution.
11. Deadlock detection and recovery.
12. Pygame visualization.
13. JSON/CSV logging.
14. Scenario runners.
15. Strategy comparison tooling.

The implementation realizes the SRS and SDD at the simulation level. It provides enough technical depth to evaluate agent strategies scientifically and enough visualization support to demonstrate behavior clearly.

Chapter 4: Agent Logic & Action Strategies

4.1 Overview

This chapter describes the decision-making logic used by the warehouse robot agents. The goal is to make the agent behavior explicit, traceable, and technically defensible.

The final system includes two strategy levels:

1. Naive baseline strategy
2. Advanced engineered strategy

The naive baseline strategy is used as a comparison point. It assigns tasks greedily to the nearest idle agent and uses standard A* without future reservations, edge-swap prevention, congestion-aware bidding, or deadlock recovery.

The advanced strategy is the main system design. It uses deterministic auction-based task allocation, congestion-aware bidding, Cooperative A* path planning, time-space reservations, priority-based conflict resolution, dynamic replanning, and deadlock recovery.

The SDD defines the robot agent as a hybrid deliberative-reactive agent. The deliberative layer handles task reasoning, goal selection, bidding, and path planning. The reactive layer handles local conflicts, blocked cells, safety overrides, and immediate replanning.

This hybrid structure is important because warehouse agents must plan ahead while still reacting safely to changing local conditions.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.2 Agent Decision Model

Each agent follows a simplified BDI-style reasoning model.

BDI means:

1. Beliefs: what the agent currently knows.
2. Desires: what the agent wants to achieve.
3. Intentions: what the agent is currently committed to doing.

The final system does not implement a full formal BDI interpreter. Instead, it implements a practical BDI-inspired decision loop suitable for simulation.

4.2.1 Beliefs

Each robot maintains operational beliefs about:

1. Its current position.
2. Its current state.
3. The static warehouse layout.
4. Pickup and drop-off locations.
5. Obstacles.
6. Assigned task, if any.
7. Available unassigned tasks.
8. Current path.
9. Future reservations.
10. Nearby or blocking agents.
11. Waiting time.
12. Replanning count.
13. Deadlock participation.
14. Local congestion.

The SDD defines similar beliefs, including current position, static map, assigned task, available tasks, nearby movements, reserved cells, congestion, and blocking status.

4.2.2 Desires / Goals

Each agent seeks to:

1. Complete assigned pickup-and-delivery tasks.
2. Minimize task completion time.
3. Avoid collisions.
4. Avoid edge swaps.
5. Avoid unnecessary waiting.
6. Avoid deadlocks.
7. Reduce congestion impact.
8. Maintain productivity when tasks are available.
9. Support the global objective of maximizing throughput.

These goals align with the SRS agent goals: complete assigned tasks, minimize completion time, avoid collisions, avoid congestion/deadlocks, and maintain utilization when tasks are available.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.2.3 Intentions / Current Plan

At any timestep, the agent may be committed to one of the following intentions:

1. Remain idle.
2. Bid for a task.
3. Move toward pickup.
4. Pick up an item.
5. Move toward drop-off.
6. Deliver an item.
7. Wait because of a conflict.
8. Replan because of blockage or congestion.
9. Participate in deadlock recovery.

The agent's intention is not fixed forever. It can change when the task state changes, the path becomes blocked, a reservation conflict occurs, or a deadlock is detected.

4.3 Agent State Machine

The final implementation uses a finite-state model for each robot.

State	Meaning
IDLE	Agent has no task and waits for assignment.
ASSIGNED	Agent owns a task and prepares movement.
MOVING_TO_PICKUP	Agent moves toward the task pickup location.
PICKING_UP	Agent collects the item at the pickup location.
MOVING_TO_DROPOFF	Agent carries the item toward the drop-off location.
DELIVERING	Agent completes delivery.
WAITING	Agent waits because of conflict, blockage, or reservation.
REPLANNING	Agent computes a new path.
DEADLOCK_RECOVERY	Agent participates in deadlock resolution.

Table 11: Agent state machine states and their responsibilities.

The SDD defines the same state-machine concept, including IDLE, BIDDING, ASSIGNED, MOVING_TO_PICKUP, PICKING_UP, MOVING_TO_DROPOFF, DELIVERING, WAITING, REPLANNING, and DEADLOCK_RECOVERY.

The implementation may represent bidding as an auction event rather than a long-lived visual state. This is an implementation simplification, not a behavioral change. Agents still evaluate tasks, submit bids, and receive deterministic task assignments.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.4 High-Level Advanced Agent Decision Cycle

The advanced agent decision cycle is executed once per simulation timestep.

Pseudo-code:

Algorithm 1: Advanced Agent Decision Cycle

Input:

- agent a_i
- current grid G
- task pool T
- reservation table R
- current timestep t

1. Update agent beliefs:
 - read current position
 - read current task state
 - read available tasks
 - read reservations
 - read congestion estimate
 - read blocking/waiting status
2. If a_i has no assigned task:
 - participate in auction if unassigned tasks exist
 - otherwise remain idle
3. If a_i receives a task assignment:
 - set current goal to pickup location
 - compute path using Cooperative A*
 - reserve future path cells and edges
4. If a_i reaches pickup location:
 - perform PICKUP
 - set current goal to drop-off location
 - compute path to drop-off
 - reserve future path cells and edges
5. If a_i reaches drop-off location:
 - perform DELIVER
 - mark task as completed
 - release remaining reservations
 - become IDLE
6. If a_i is moving:
 - select next cell from path
 - submit movement intention
7. Check proposed movement:
 - if next cell is invalid:

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

wait or replan
if movement conflicts with another agent:
 apply priority-based conflict resolution
if reservation conflict exists:
 wait or replan

8. If ai waits too long:
 trigger replanning

9. If deadlock is detected:
 participate in deadlock recovery

10. Execute approved action:
 move, wait, pickup, deliver, or replan

11. Update logs and metrics.

This cycle follows the SDD agent coordination step, where agents update beliefs, participate in auctions, plan paths, reserve cells, broadcast movement intentions, resolve conflicts, wait or replan, participate in deadlock recovery, execute an action, and update logs.

4.5 Naive Baseline Strategy

The naive baseline strategy is intentionally simple. It is used to show what happens when agents do not use the advanced coordination mechanisms.

The naive baseline uses:

1. Nearest idle agent task assignment.
2. Standard A* path planning.
3. Current occupancy checking.
4. Waiting when blocked.
5. No auction bidding.
6. No congestion-aware bid cost.
7. No Cooperative A*.
8. No reservation table.
9. No edge-swap prevention.
10. No explicit deadlock recovery.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Pseudo-code:

Algorithm 2: Naive Baseline Strategy

Input:

task pool T
agent set A
grid G

For each unassigned task T_k :

1. Find all idle agents.
2. For each idle agent a_i :
 compute Manhattan distance from a_i to T_k .pickup.
3. Select agent with minimum distance.
4. If tie:
 select lowest agent_id.
5. Assign T_k to selected agent.

For each assigned agent a_i :

1. If a_i has no path:
 run standard A* from current position to current goal.
2. Select next cell from path.
3. If next cell is free and valid:
 move to next cell.
4. Else:
 wait.

The naive baseline is expected to perform worse in dense multi-agent conditions because it makes locally reasonable decisions without future coordination. It does not reserve future cells, does not prevent edge swaps, and does not recover from deadlocks.

In validation, the naive baseline completed only 12 tasks in the baseline scenario, while the advanced strategy completed 42 tasks. The naive baseline also produced one actual collision and negative social welfare, while the advanced strategy produced zero collisions and positive social welfare.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.6 Advanced Auction-Based Task Allocation

The advanced strategy uses deterministic auction-based task allocation. This prevents multiple agents from claiming the same task and helps distribute work based on estimated cost rather than only nearest distance.

For each task, every idle candidate agent computes a bid.

The bid cost is:

$$\begin{aligned} \text{BidCost}_i(\text{Tk}) = & \alpha \times \text{Distance}_i(\text{Tk}) \\ & + \beta \times \text{Congestion}_i(\text{Tk}) \\ & + \gamma \times \text{Workload}_i \\ & + \delta \times \text{ReservationDelay}_i(\text{Tk}) \end{aligned}$$

Where:

Term	Meaning
Distance _i (Tk)	Estimated travel distance from agent i to pickup and then to drop-off.
Congestion _i (Tk)	Estimated congestion along the expected route.
Workload _i	Cost representing whether the agent is idle or busy.
ReservationDelay _i (Tk)	Estimated delay caused by existing reservations.
$\alpha, \beta, \gamma, \delta$	Weights controlling the importance of each component.

Table 12: Auction-based task allocation process used by the advanced strategy. The SDD defines the same bid-cost model using distance, congestion, workload, reservation delay, and tunable weights.

Algorithm 3: Deterministic Auction-Based Task Allocation

Input:

unassigned tasks T_{unassigned}
agent set A
current timestep t

For each task Tk in T_{unassigned}:

1. Create empty bid list B.
2. For each agent ai in A:
 - If ai is IDLE:
 - distance_cost = distance(ai.position, Tk.pickup) + distance(Tk.pickup, Tk.dropoff)
 - congestion_cost = estimate congestion along expected route
 - workload_cost = 0 if ai is idle, high value otherwise
 - reservation_delay = estimate delay caused by reserved cells

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

$$\text{bid} = \alpha * \text{distance_cost} + \beta * \text{congestion_cost} + \gamma * \text{workload_cost} + \delta * \text{reservation_delay}$$

Add (ai, bid) to B.

3. If B is empty:
keep task unassigned.
4. Else:
winner = agent with lowest bid.
5. If multiple agents have equal lowest bid:
winner = agent with lowest agent_id.
6. Assign Tk to winner.
7. Set Tk.state = ASSIGNED.
8. Set winner.assigned_task = Tk.
9. Log bid submissions and assignment result.

The deterministic tie-breaker is critical. It prevents nondeterministic results and ensures that the same scenario with the same seed produces the same assignment sequence.

4.7 Why Auction Allocation Is Better Than Nearest-Agent Assignment

Nearest-agent assignment is simple but shortsighted.

It ignores:

1. Whether the nearest agent is likely to enter a congested route.
2. Whether another agent has a clearer future path.
3. Whether reservations will delay the nearest agent.
4. Whether assigning the nearest agent overloads one region of the warehouse.
5. Whether task distribution remains balanced.

The auction strategy improves this by including distance, congestion, workload, and reservation delay in one decision score.

This does not guarantee global optimality. However, it gives a practical decentralized decision rule that is deterministic, cheap to compute, and effective in simulation.

In the validation comparison, the advanced strategy had fewer invalid move preventions and much higher utility than the naive baseline. This supports the claim that the auction and coordination mechanisms improve practical performance rather than only adding theoretical complexity.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.8 Cooperative A* Strategy

The advanced path planner uses Cooperative A*. Unlike standard A*, Cooperative A* plans over both space and time.

A normal A* state is:

(row, col)

A Cooperative A* state is:

(row, col, timestep)

This difference is essential. In a multi-agent system, a cell is not simply free or blocked forever. A cell may be safe at timestep 10 but unsafe at timestep 11 because another agent has reserved it.

The SDD explicitly defines Cooperative A* as a planner that searches over cell plus timestep states and avoids reserved cells and edge conflicts.

Pseudo-code:

Algorithm 4: Cooperative A* Path Planning

Input:

start cell s
goal cell g
current timestep t0
grid G
reservation table R

Output:

path P or failure

1. Create start node:
node = (s.row, s.col, t0)
2. Insert start node into open set.
3. Initialize closed set as empty.
4. While open set is not empty:
current = node with lowest f-cost
If current.cell == goal:
return reconstructed path

Add current to closed set.

For each action in {UP, DOWN, LEFT, RIGHT, WAIT}:

next_cell = apply action to current.cell
next_time = current.time + 1

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

If next_cell is outside grid:
 reject successor

If next_cell is obstacle:
 reject successor

If next_cell is reserved at next_time:
 reject successor

If edge from current.cell to next_cell is reserved
in opposite direction at next_time:
 reject successor

If successor already in closed set:
 reject successor

$g_cost = current.g_cost + 1$
 $h_cost = \text{ManhattanDistance}(next_cell, goal)$
 $congestion_penalty = \text{congestion around } next_cell$

$f_cost = g_cost + h_cost + congestion_penalty$

Add successor to open set.

5. If no path is found:
 return failure.

The WAIT action is included as a legal action. This is necessary because narrow corridors often require one agent to wait while another passes. Without WAIT, the planner would falsely fail in situations that are actually solvable.

The SDD also emphasizes that wait must be included as a valid action because some narrow-corridor situations require it.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.9 Reservation Table Strategy

The reservation table is the core mechanism that allows Cooperative A* to prevent future conflicts.

It stores:

1. Which agent reserved a cell.
2. Which cell is reserved.
3. At which timestep the cell is reserved.
4. Which edge movement is reserved.
5. When the reservation expires.

Two reservation types are used:

Reservation Type	Purpose
Vertex reservation	Prevent two agents from occupying the same cell at the same timestep.
Edge reservation	Prevent two agents from swapping cells across the same edge at the same timestep.

Table 13: Reservation-table rules used to prevent vertex and edge-swap conflicts.

Pseudo-code:

Algorithm 5: Reserve Planned Path

Input:

agent a_i
path P
reservation table R

1. Release previous future reservations owned by a_i .

2. For each step k in path P :

cell = $P[k]$
time = current_time + k

Add vertex reservation:

$R.reserve_vertex(a_i, cell, time)$

3. For each consecutive pair in path P :

from_cell = $P[k]$
to_cell = $P[k + 1]$
time = current_time + $k + 1$

Add edge reservation:

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

R.reserve_edge(ai, from_cell, to_cell, time)

4. Remove expired reservations from R.

The reservation table is what makes the advanced strategy safer than the naive baseline. The naive baseline only reacts to the current occupancy state. The advanced strategy prevents agents from planning conflicting future movements before the conflict actually occurs. The SDD identifies this component as essential for Cooperative A* and future collision prevention.

4.10 Movement Intention and Conflict Detection

Before applying movement, the simulator checks intended actions.

Each moving agent proposes a next cell. The conflict resolver checks whether proposed actions are compatible.

Conflict types:

Conflict Type	Condition	Example
Vertex conflict	Two agents want the same next cell.	A1 and A2 both move to (4,5).
Edge conflict	Two agents swap positions.	A1 moves from (4,5) to (4,6), while A2 moves from (4,6) to (4,5).
Blocking conflict	Agent wants to enter a cell occupied by an agent that is waiting.	A1 wants to enter A2's cell, but A2 does not move.
Invalid movement	Agent tries to move outside grid, into obstacle, or into forbidden cell.	A1 moves into blocked cell.
Reservation conflict	Agent attempts to move into a reserved future cell or edge.	A1 violates reservation table.

Table 14: Movement conflict types checked before agent actions are executed.

The SDD defines vertex, edge, and blocking conflicts as the main movement conflict types.

Pseudo-code:

Algorithm 6: Movement Conflict Detection

Input:

agent movement intentions I
current positions P
reservation table R

Output:

conflict list C

1. Initialize C as empty.
2. For each agent ai:

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

proposed_next = I[ai]

If proposed_next is outside grid:
add INVALID_MOVE to C

If proposed_next is obstacle:
add INVALID_MOVE to C

If proposed_next is reserved at next timestep:
add RESERVATION_CONFLICT to C

3. For each pair of agents (ai, aj):

If ai.next_cell == aj.next_cell:
add VERTEX_CONFLICT(ai, aj)

If ai.current_cell == aj.next_cell
and ai.next_cell == aj.current_cell:
add EDGE_CONFLICT(ai, aj)

If ai.next_cell == aj.current_cell
and aj.action == WAIT:
add BLOCKING_CONFLICT(ai, aj)

4. Return C.

The movement conflict detector works as a final safety gate. Even if an earlier planner or reservation check fails, an unsafe move can still be blocked before execution.

4.11 Priority-Based Conflict Resolution

When a conflict occurs, the advanced strategy does not resolve it randomly. It uses deterministic priority.

Priority is based on:

1. Carrying item status.
2. Waiting time.
3. Progress toward task completion.
4. Agent ID tiebreaker.

Informally:

carrying item > longer waiting time > closer to completion > lower agent ID

Pseudo-code:

Algorithm 7: Priority-Based Conflict Resolution

Input:

conflict list C
agent set A

For each conflict c in C:

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

1. Identify involved agents.
2. For each involved agent a_i :
 $\text{priority}(a_i) =$
 $\lambda_1 * \text{carrying_item}(a_i)$
 $+ \lambda_2 * \text{waiting_time}(a_i)$
 $+ \lambda_3 * \text{progress_to_goal}(a_i)$
 $- \lambda_4 * \text{agent_id}(a_i)$
3. winner = agent with highest priority.
4. winner keeps intended movement if still valid.
5. All losing agents:
 cancel movement
 execute WAIT
 increment waiting_time
6. If losing agent waited longer than WAIT_THRESHOLD:
 trigger replanning.
7. Log conflict and resolution.

The SDD defines the same priority logic: carrying item, waiting time, progress, and lower agent ID as a deterministic tiebreaker.

This design prevents unstable behavior. Random conflict resolution might look acceptable in one run and fail in another. Deterministic priority keeps the system reproducible and explainable.

4.12 Dynamic Replanning Strategy

Agents replan when the current path is no longer safe or efficient.

Replanning triggers:

1. Next cell is blocked.
2. Reservation conflict occurs.
3. Agent loses repeated conflicts.
4. Agent waits longer than the waiting threshold.
5. Congestion increases significantly.
6. Deadlock is detected.
7. Path becomes invalid.
8. Agent reaches pickup and must switch goal to drop-off.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Pseudo-code:

Algorithm 8: Dynamic Replanning

Input:

agent ai
current position pos
current goal goal
reservation table R

1. Release future reservations owned by ai.
2. Run Cooperative A*:
 new_path = CooperativeAStar(pos, goal, R)
3. If new_path exists:
 ai.path = new_path
 reserve new_path in R
 reset waiting counter
 set ai.state = moving state
 return SUCCESS
4. Else:
 try fallback A* or wait temporarily
5. If fallback also fails:
 ai.action = WAIT
 ai.waiting_time += 1
 retry next timestep
6. Log replanning event.

The SDD defines dynamic replanning as a response to blocked cells, rejected reservations, congestion, waiting thresholds, deadlocks, conflict warnings, and invalid paths.

Replanning is necessary because the environment is dynamic. A path that was safe several timesteps earlier may become inefficient or blocked as other agents move and new tasks appear.

4.13 Deadlock Detection Strategy

A deadlock occurs when agents are waiting for each other in a cycle.

Example:

A1 waits for A2

A2 waits for A3

A3 waits for A1

This forms a wait-for graph cycle:

A1 → A2 → A3 → A1

The final implementation uses persistent waiting/blocking before declaring a true deadlock. This prevents ordinary short waits from being misclassified as deadlocks.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Pseudo-code:

Algorithm 9: Deadlock Detection

Input:

agent set A
waiting relationships W
persistence threshold K

Output:

deadlock cycles D

1. Initialize directed graph WFG.
2. For each agent a_i in A:
 If a_i is waiting because of agent a_j :
 Add edge $a_i \rightarrow a_j$.
3. For each edge $a_i \rightarrow a_j$:
 Update persistence counter for this waiting relation.
4. Remove edges that are not persistent enough.
5. Run cycle detection on WFG.
6. If cycle exists:
 D = agents involved in the cycle
 return D
7. Else:
 return no deadlock.

The SDD defines wait-for graph deadlock detection exactly: nodes are agents, edges represent waiting relationships, and a cycle means a deadlock exists.

The persistence threshold is an implementation improvement. It avoids inflating deadlock counts with harmless one-step waits.

4.14 Deadlock Recovery Strategy

When a deadlock is detected, the system breaks the cycle using deterministic recovery.

Recovery steps:

1. Identify agents in the deadlock cycle.
2. Freeze involved agents for a short recovery step.
3. Compute priority for each involved agent.
4. Let the highest-priority agent keep its reservation if valid.
5. Lower-priority agents release future reservations.
6. Lower-priority agents replan.
7. If replanning succeeds, movement resumes.
8. If replanning fails, the agent waits and retries.
9. Continue until the cycle disappears.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Pseudo-code:

Algorithm 10: Deadlock Recovery

Input:

deadlock cycle D
reservation table R
agent set A

1. Identify involved agents in D.
2. For each agent a_i in D:
temporarily freeze movement for recovery step.
3. Compute $\text{priority}(a_i)$ for each involved agent.
4. a_{high} = highest-priority agent.
5. For each agent a_i in D where $a_i \neq a_{\text{high}}$:
release future reservations of a_i from R
set $a_i.\text{state} = \text{REPLANNING}$
6. For each replanning agent a_i :
new_path = CooperativeAStar($a_i.\text{position}$, $a_i.\text{goal}$, R)

If new_path exists:
 $a_i.\text{path} = \text{new_path}$
 reserve new_path
 $a_i.\text{state} = \text{moving state}$
Else:
 $a_i.\text{action} = \text{WAIT}$
7. Rebuild wait-for graph.
8. If no cycle remains:
log DEADLOCK_RESOLVED.
9. Else:
repeat recovery in deterministic order.

The SDD defines deadlock recovery using freezing, priority selection, reservation release, replanning, and deterministic waiting if no path exists.

The important engineering point is that deadlocks are not treated as fatal errors. They are expected stress events in dense multi-agent systems. The strategy is successful if deadlocks are detected and resolved without causing collisions or halting the simulation.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.15 Congestion-Aware Strategy

Congestion is handled in two places:

1. During task allocation.
2. During path planning and replanning.

In task allocation, congestion increases the bid cost. This discourages assigning tasks to agents whose expected route passes through crowded regions.

In path planning, congestion can increase the cost of moving through crowded cells. This makes alternative routes more attractive when available.

Congestion-aware behavior is important because a collision-free system can still perform badly if agents constantly block each other in narrow corridors.

The OnePager identifies congestion-aware auction-based task allocation as one of the core strategies, where agents bid using distance, congestion, and workload.

Pseudo-code:

Algorithm 11: Congestion-Aware Cost Adjustment

Input:

candidate cell c
current timestep t
local agent positions A

1. Count agents near c :
 $\text{nearby_count} = \text{number of agents within local radius of } c$
2. Estimate local capacity:
 $\text{capacity} = \text{expected safe number of agents near } c$
3. Compute congestion:
 $\text{congestion}(c, t) = \text{nearby_count} / \text{capacity}$
4. If congestion is high:
 increase path cost through c
5. Use congestion value in:
 task bid calculation
 Cooperative A* successor cost
 replanning decision

This strategy reduces unnecessary crowding but does not guarantee that congestion disappears. In dense scenarios, congestion is expected. The goal is to keep the system safe and productive despite it.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.16 Pickup and Delivery Strategy

Each task has two movement phases:

1. Move to pickup.
2. Move to drop-off.

Pseudo-code:

Algorithm 12: Pickup and Delivery Execution

Input:

agent ai
assigned task Tk

1. If ai has assigned task and is not carrying item:
goal = Tk.pickup

2. If ai.position == Tk.pickup:
execute PICKUP
ai.carrying_item = true
goal = Tk.dropoff
release old reservations
plan path to drop-off
reserve new path

3. If ai.carrying_item == true:
continue moving toward Tk.dropoff

4. If ai.position == Tk.dropoff:
execute DELIVER
Tk.state = COMPLETED
Tk.completion_time = current timestep
ai.carrying_item = false
ai.assigned_task = none
release remaining reservations
ai.state = IDLE

5. Log pickup and delivery events.

This satisfies the SRS requirement that an agent assigned to a task must navigate from its current position to the pickup location and then to the drop-off location.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.17 Utility and Social Welfare Strategy

The agents are evaluated using utility and social welfare metrics.

The utility function used in validation is:

$$U_i = 10 \times \text{completed_tasks}_i - 0.05 \times \text{waiting_time}_i - 0.10 \times \text{replanning_count}_i \\ - 0.05 \times \text{conflicts_involved}_i - 1.00 \times \text{deadlocks_involved}_i$$

Social welfare is:

$$\text{Social Welfare} = \sum U_i$$

Average utility is:

$$\text{Average Utility} = \text{Social Welfare} / \text{Number of Agents}$$

The utility function rewards productive work and penalizes coordination overhead.

The reward of 10 for completed tasks ensures that completing orders remains the dominant goal. Waiting, replanning, and conflicts receive smaller penalties because they are normal coordination costs in dense MAS environments. Deadlock involvement receives a stronger penalty because deadlocks represent more severe coordination failures.

This matches the SRS idea that system performance should be evaluated using utility and social welfare, where the system aims to maximize social welfare while maintaining efficient coordination.

Pseudo-code:

Algorithm 13: Utility and Social Welfare Calculation

Input:

agent metrics M
agent set A

1. social_welfare = 0

2. For each agent a_i in A:

```
utility_i =  
    10 * tasks_completed_by_agent(ai)  
    - 0.05 * waiting_time(ai)  
    - 0.10 * replanning_count(ai)  
    - 0.05 * conflicts_involved(ai)  
    - 1.00 * deadlocks_involved(ai)
```

```
store utility_i
```

```
social_welfare += utility_i
```

3. average_utility = social_welfare / number_of_agents

4. Export utility, social welfare, and average utility.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The utility function is used mainly for evaluation and comparison. It is not the only internal decision rule used by agents. Internally, agents use bid costs, path costs, reservations, priority rules, and replanning triggers.

This is a reasonable design because utility summarizes final performance, while operational decisions must remain local and computationally cheap.

4.18 Strategy Selection in the Simulator

The final implementation supports strategy selection from the command line.

Available strategies:

1. advanced
2. naive

Example:

- `python run_simulation.py --scenario baseline --strategy advanced --steps 300 --out runs/baseline_advanced`
- `python run_simulation.py --scenario baseline --strategy naive --steps 300 --out runs/baseline_naive`

The strategy mode affects:

1. Task allocation.
2. Path planning.
3. Reservation usage.
4. Conflict prevention.
5. Deadlock handling.
6. Metrics and comparison results.

The comparison runner executes both strategies on the baseline scenario:

```
python run_strategy_comparison.py
```

This produces a direct naive-vs-advanced comparison file.

The strategy comparison is important because the final report guidelines require utility and social welfare charts compared to baseline strategies.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.19 Full Advanced Strategy Flow

The complete advanced strategy combines all previously described mechanisms.

Pseudo-code:

Algorithm 14: Full Advanced Strategy

Input:

- warehouse grid G
- agent set A
- task pool T
- reservation table R
- current timestep t

1. Generate new tasks and add them to T.
2. For each unassigned task:
 - run deterministic auction:
 - collect bids from idle agents
 - choose lowest bid
 - break ties using lowest agent_id
 - assign task to winner
3. For each assigned agent:
 - If agent has no valid path:
 - run Cooperative A*
 - reserve future vertices and edges
4. For each moving agent:
 - compute next intended cell
5. Detect movement conflicts:
 - vertex conflicts
 - edge conflicts
 - blocking conflicts
 - reservation conflicts
 - invalid moves
6. Resolve conflicts:
 - compute priority
 - allow winner to move
 - force losing agents to wait or replan
7. Detect persistent deadlocks:
 - build wait-for graph
 - search for cycles
8. If deadlock exists:
 - select highest-priority agent

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

- release lower-priority reservations
- replan lower-priority agents
- continue when cycle is broken

9. Execute all approved actions synchronously.

10. Update:

- agent positions
- task states
- reservations
- metrics
- logs
- visualization

11. Repeat until simulation ends.

This is the core engineered strategy of the project.
It directly addresses the main MAS coordination problems:

Problem	Strategy Mechanism
Multiple agents want same task	Auction-based task allocation
Nearest agent may not be best	Congestion-aware bid cost
Agents may collide	Reservation table and conflict detection
Agents may swap positions	Edge reservations
Agents may block each other	Priority-based waiting and replanning
Agents may deadlock	Wait-for graph detection and recovery
Dense traffic reduces efficiency	Congestion-aware path costs and replanning
Validation needs evidence	Logs, metrics, utility, social welfare, GUI

Table 15: Strategy mechanisms used to address the main MAS coordination problems. This aligns with the SDD summary, which connects multiple agents wanting the same task to auctions, overloaded routes to congestion-aware costs, collisions to movement intentions and reservations, edge swaps to edge reservations, narrow corridors to priority rules, cyclic blocking to wait-for graph deadlock detection, and dynamic problems to replanning.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

4.20 Why the Advanced Strategy Is Suitable

The advanced strategy is suitable because it addresses both long-term planning and short-term safety.

A purely greedy strategy is too weak. It can assign tasks quickly, but it cannot reason about future conflicts, reservations, edge swaps, or deadlocks.

A purely offline centralized planner would also be inappropriate for this project because tasks arrive dynamically and the goal is to model decentralized-style coordination between agents.

The selected strategy balances practicality and MAS relevance:

1. Auctions provide decentralized-style task allocation.
2. Cooperative A* provides future-aware path planning.
3. Reservations prevent hidden future conflicts.
4. Priority rules make conflict resolution deterministic.
5. Replanning allows adaptation to dynamic conditions.
6. Wait-for graphs detect cyclic blocking.
7. Metrics and logs make the behavior auditable.
8. The naive baseline provides a meaningful comparison.

The validation results support this design choice. Compared with the naive baseline, the advanced strategy completed more tasks, avoided actual collisions, reduced invalid move attempts, and achieved much higher social welfare.

4.21 Chapter Summary

This chapter presented the decision-making logic and action strategies used in the final implementation.

The system includes a naive baseline and an advanced engineered strategy. The naive baseline uses nearest-agent assignment and standard A*. The advanced strategy uses auction-based task allocation, congestion-aware bidding, Cooperative A*, time-space reservations, priority-based conflict resolution, dynamic replanning, and deadlock recovery.

The main agent decision loop is BDI-inspired but implemented pragmatically through state machines, path planning, reservations, conflict checks, and logging. The advanced strategy is deterministic and reproducible, which is essential for simulation validation.

The key conclusion is that the advanced strategy is not merely more complex than the naive baseline. It solves real multi-agent coordination problems that the naive strategy cannot handle reliably. The validation results show that these mechanisms improve productivity, safety, utility, and social welfare.

Chapter 5: Simulator & Visualization Platform

5.1 Overview

The simulator and visualization platform are the user-facing parts of the project. Their purpose is not only to animate the system, but also to make the multi-agent behavior observable, testable, and explainable.

The simulator executes the warehouse MAS in discrete timesteps. At each timestep, agents receive tasks, plan paths, reserve cells, move, wait, replan, detect conflicts, resolve deadlocks, and update metrics.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The visualization platform displays this behavior in real time using a Pygame dashboard. It allows an observer to inspect the warehouse grid, agents, pickup/drop-off stations, tasks, planned paths, reservations, congestion, event logs, and live performance metrics. This chapter explains how to run the simulator, how to use the dashboard, what each visual element means, and how the exported logs support validation.

5.2 Simulator Purpose

The simulator has four main purposes:

1. Execute the multi-agent warehouse system under controlled conditions.
2. Demonstrate agent behavior visually.
3. Generate quantitative results for validation.
4. Provide reproducible evidence through exported logs and metrics.

The simulator is designed for repeatable experiments, not physical robot deployment. This means that the system focuses on coordination logic, path planning, conflict prevention, deadlock handling, task completion, and performance measurement.

The main validation advantage of the simulator is that every important event can be observed or logged. This includes task creation, task assignment, path planning, movement, pickup, delivery, waiting, invalid move prevention, deadlock detection, and deadlock recovery.

5.3 Simulator Execution Model

The simulator uses a synchronous discrete-time execution model.

At every timestep:

1. New tasks may be generated.
2. Idle agents may receive or bid for tasks.
3. Assigned agents plan or follow paths.
4. Agents propose movement intentions.
5. The system checks conflicts and reservations.
6. Unsafe moves are blocked before execution.
7. Deadlocks are detected if persistent blocking occurs.
8. Agents execute one approved action.
9. Task states are updated.
10. Metrics and logs are recorded.
11. The GUI renders the new state if rendering is enabled.

This model is appropriate for the warehouse domain because robot movement is naturally sequential and step-based. Each robot can move one grid cell per timestep, wait, pick up an item, deliver an item, or replan.

The synchronous model also makes validation clearer. Since all agents act once per timestep, it is easy to identify when a conflict was prevented and which agent was allowed to move.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.4 Running the Simulator

The simulator is executed from the command line.

The main script is:

`run_simulation.py`

General command format:

```
python run_simulation.py --scenario SCENARIO_NAME --strategy STRATEGY_NAME --steps  
NUMBER_OF_STEPS --out OUTPUT_FOLDER
```

Example advanced baseline run:

```
python run_simulation.py --scenario baseline --strategy advanced --steps 300 --out  
runs/baseline_advanced
```

Example naive baseline run:

```
python run_simulation.py --scenario baseline --strategy naive --steps 300 --out  
runs/baseline_naive
```

Example GUI run:

```
python run_simulation.py --scenario baseline --strategy advanced --steps 600 --render --out  
runs/baseline_gui
```

Example long high-load stress run:

```
python run_simulation.py --scenario high_load --strategy advanced --steps 6000 --out  
runs/high_load_6000
```

Run all predefined advanced validation scenarios:

```
python run_all_scenarios.py
```

Run direct naive-vs-advanced comparison:

```
python run_strategy_comparison.py
```

5.5 Installation Requirements

The implementation uses Python 3.x.

The main external dependency is Pygame for visualization.

Install dependencies using:

```
pip install -r requirements.txt
```

If Pygame is not already installed:

```
pip install pygame
```

The simulator can run without rendering. This is useful for validation because headless runs are faster and produce the same output files.

The GUI requires the `--render` flag.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.6 Supported Scenarios

The simulator supports the following scenarios:

Scenario	Purpose
baseline	Tests normal operating conditions.
high_load	Tests frequent task generation and saturation behavior.
high_density	Tests many agents in the same warehouse grid.
deadlock	Tests blocking and deadlock-prone movement.
dynamic	Tests dynamic task behavior under moderate pressure.
scalability	Tests increased agent count and system-level throughput.
long high-load	Tests extended stability over 6000 timesteps.

Table 16: Supported simulation scenarios available in the final visualization platform. Each scenario is deterministic when the same random seed is used. This makes the results reproducible and suitable for report validation.

5.7 Strategy Selection

The simulator supports two strategy modes:

Strategy	Description
naive	Greedy nearest-agent assignment with standard A* and no reservation table.
advanced	Auction-based assignment, Cooperative A*, reservation table, conflict prevention, replanning, and deadlock recovery.

Table 17: strategy selection modes.

The strategy is selected using the `--strategy` argument.

Example:

- `python run_simulation.py --scenario baseline --strategy advanced --steps 300 --out runs/baseline_advanced`
- `python run_simulation.py --scenario baseline --strategy naive --steps 300 --out runs/baseline_naive`

This is important because the final validation compares the advanced strategy against a real implemented baseline, not only against a theoretical baseline.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.8 Dashboard Layout

The GUI dashboard is organized to show both spatial behavior and numeric validation data.

The dashboard contains:

1. Main warehouse grid.
2. Agents and their current positions.
3. Pickup and drop-off locations.
4. Obstacles.
5. Assigned and unassigned task indicators.
6. Planned paths.
7. Future reservations.
8. Congestion heatmap.
9. Live metrics panel.
10. Agent state panel.
11. Event log.
12. Legend and keyboard controls.



Figure 2: Real-time simulation dashboard showing the warehouse grid, agents, tasks, paths, reservations, and live metrics.

The GUI is especially useful for debugging and demonstration. It allows an observer to confirm that agents are not passing through obstacles, not occupying the same cell, not edge-swapping, and not getting stuck permanently in deadlock cycles.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.9 Grid Visualization

The warehouse is shown as a two-dimensional grid.

Grid Element	Visual Meaning
Free cell	Traversable warehouse space.
Obstacle	Blocked cell that agents cannot enter.
Pickup station	Location where tasks begin.
Drop-off station	Location where tasks are delivered.
Robot agent	Autonomous warehouse robot.
Planned path	Agent route toward pickup or drop-off.
Reserved cell	Future cell reserved by an agent.
Congested region	Area with high local agent density.
Conflict marker	Location where an invalid move or conflict was detected.
Deadlock marker	Area or agents involved in deadlock recovery.

Table 18: Grid visualization elements and their visual meaning.

This visual encoding makes the environment understandable at a glance. The observer can immediately see whether agents are navigating correctly, whether paths avoid obstacles, and whether traffic accumulates in bottleneck regions.

5.10 Agent Visualization

Each agent is visually identifiable.

The dashboard displays:

1. Agent ID.
2. Agent position.
3. Agent shape or icon.
4. Direction of movement.
5. Current state.
6. Whether the agent is carrying an item.
7. Planned path.
8. Task assignment, when relevant.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Agent states include:

State	Meaning
IDLE	Agent has no task.
ASSIGNED	Agent owns a task and prepares movement.
MOVING_TO_PICKUP	Agent is going to the pickup station.
PICKING_UP	Agent is collecting the item.
MOVING_TO_DROPOFF	Agent is carrying the item to destination.
DELIVERING	Agent is completing delivery.
WAITING	Agent is blocked or yielding.
REPLANNING	Agent is computing a new path.
DEADLOCK_RECOVERY	Agent is involved in deadlock resolution.

Table 19: Agent states displayed in the simulator dashboard.

This matters because a simulation with moving dots is not enough for validation. The observer must understand what each robot is doing and why.

5.11 Task Visualization

Tasks are visualized through pickup and drop-off locations and, when enabled, task links.

Task State	Visualization
UNASSIGNED	Task exists but has no assigned robot.
ASSIGNED	Task is linked to the assigned agent.
COMPLETED	Task is removed from active view and counted in metrics.

Table 20: Task states displayed in the simulator dashboard.

The task lifecycle is:

UNASSIGNED → ASSIGNED → COMPLETED

The GUI helps verify that tasks are not duplicated and that each active task is handled by one agent at a time.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.12 Path and Reservation Visualization

The advanced strategy depends heavily on Cooperative A* and reservations. Therefore, the GUI includes optional overlays for paths and reservations.

Path visualization shows where agents intend to move.

Reservation visualization shows future cells or edges reserved by agents.

These overlays are important because they expose the internal coordination mechanism.

Without path and reservation visualization, the observer could see that agents move safely but would not understand why they are safe.

The reservation overlay helps verify that agents are not simply reacting at the last moment.

They are planning future movement and avoiding conflicts before they occur.

5.13 Congestion Visualization

The dashboard includes a congestion heatmap.

Congestion represents local crowding in the warehouse. It is especially useful in high-load, high-density, and scalability scenarios.

Congestion visualization helps answer three questions:

1. Where are the bottlenecks?
2. Are too many agents competing for the same corridor or station?
3. Is high throughput coming at the cost of poor traffic flow?

The congestion heatmap is not only a visual feature. It supports validation because high-density scenarios are expected to increase congestion, and the dashboard makes this visible.

5.14 Conflict and Invalid Move Visualization

The simulator distinguishes between actual collisions and invalid moves that were prevented.

This distinction is important.

An actual collision means the safety mechanism failed.

An invalid move prevented means the safety mechanism detected a dangerous action and blocked it before execution.

The dashboard and logs can show:

1. Vertex conflicts.
2. Edge-swap conflicts.
3. Blocking conflicts.
4. Reservation conflicts.
5. Invalid movement attempts.
6. Waiting or replanning responses.

This allows the observer to verify that the system is safe because it actively prevents dangerous moves, not because no dangerous situations ever occur.

5.15 Deadlock Visualization

Deadlock-related events are shown through logs and visual indicators.

When deadlock detection is triggered, the simulator records:

1. The timestep of detection.
2. The involved agents.
3. The blocking or waiting relationship.
4. The recovery action.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5. Whether agents resumed movement.

Deadlocks are expected in difficult multi-agent scenarios, especially in narrow corridors or dense grids. The important validation condition is not that deadlocks never occur. The important condition is that deadlocks are detected, resolved, and do not cause permanent system failure.

5.16 GUI Controls

When running with `--render`, the dashboard supports interactive controls.

Control	Function
SPACE	Pause or resume the simulation.
S	Step once while paused.
R	Reset the simulation.
P	Show or hide planned paths.
V	Show or hide reservations.
H	Show or hide congestion heatmap.
T	Show or hide task links.

Table 21: GUI controls available during rendered simulation.

These controls make the simulator useful for inspection. For example, pausing and stepping one timestep at a time helps explain why an agent waited, why a move was blocked, or why a path was replanned.

5.17 Reset Behavior

The reset control restarts the simulation state.

A correct reset must rebuild:

1. Agent positions.
2. Agent states.
3. Task pool.
4. Grid occupancy.
5. Reservations.
6. Metrics.
7. Event logs.
8. Random generator state.
9. Simulation timestep.

This is important because a visual reset alone would not be enough. The simulation must return to the same initial conditions so that behavior remains reproducible.

The final implementation supports proper reset behavior.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.18 Live Metrics Panel

The live metrics panel shows current system performance during simulation.

Metric	Meaning
Time step	Current simulation tick.
Active agents	Number of agents currently operating.
Pending tasks	Tasks not yet assigned.
Assigned tasks	Tasks currently owned by agents.
Completed tasks	Finished pickup/drop-off jobs.
Throughput	Completed tasks divided by elapsed timesteps.
Average completion time	Mean time from task creation to delivery.
Invalid moves prevented	Unsafe movement attempts blocked before execution.
Deadlocks detected	Number of detected deadlock events.
Collision events	Actual collision events.
Utilization	Fraction of agent time spent active.
Congestion index	Current or average congestion pressure.

Table 22: Live metrics displayed in the simulator dashboard.

These metrics make the dashboard useful for validation and not just for demonstration.

5.19 Event Log

The simulator records important events during each run.

Main event types include:

1. TASK_CREATED
2. TASK_BID_SUBMITTED
3. TASK_ASSIGNED
4. PATH_PLANNED
5. PATH_REPLANNED
6. AGENT_MOVED
7. AGENT_WAITED
8. TASK_PICKED_UP
9. TASK_COMPLETED
10. INVALID_MOVE_PREVENTED
11. DEADLOCK_DETECTED

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

- 12. DEADLOCK_RESOLVED
- 13. SIMULATION_STARTED
- 14. SIMULATION_ENDED

The event log is important because it provides a chronological explanation of the simulation. If a result looks surprising, the event log can be inspected to understand what happened. For example, if an agent waits repeatedly, the log can show whether the cause was a blocked cell, a reservation conflict, congestion, or deadlock recovery.

5.20 Exported Output Files

Each simulation run exports structured validation files.

Output File	Purpose
summary.json	Final run metrics and agent-level summary.
events.json	Full chronological event log.
events.csv	Event log in spreadsheet-readable format.
agent_decisions.json	Agent state and decision traces.
snapshots.json	Timestep-level system snapshots.
snapshots.csv	Snapshot data in spreadsheet-readable format.
strategy_comparison_summary.json	Naive-vs-advanced comparison output.

Table 23: Exported simulator output files and their validation purpose.

These files make the experiment reproducible and auditable.

The GUI shows what happened visually. The exported files prove what happened numerically.

5.22 Observer Verification Checklist

The simulator allows an observer to verify the main requirements visually and through logs.

Requirement	How the simulator verifies it
Task generation	New tasks appear and are logged.
Task assignment	Tasks are assigned to one agent.
Task lifecycle	Tasks progress from unassigned to assigned to completed.
Collision avoidance	No two agents occupy the same cell.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Edge-swap prevention	Opposite-direction swaps are blocked.
Path planning	Agents follow planned routes.
Dynamic replanning	Paths change after conflicts or blocked movement.
Congestion handling	Heatmap shows crowded regions and agents adapt.
Deadlock recovery	Deadlocks are detected and logged as resolved.
Metrics export	JSON/CSV files are generated after each run.
Strategy comparison	Naive and advanced runs can be compared numerically.

Table 24: Observer checklist for verifying simulator behavior and project requirements. This checklist is important because it connects the simulator directly to the report's validation claims.

5.23 Relationship Between GUI and Validation Results

The GUI and exported metrics serve different but complementary roles.

The GUI provides qualitative evidence:

1. Agents move correctly.
2. Agents avoid obstacles.
3. Agents do not visually collide.
4. Agents follow paths.
5. Agents wait or replan when blocked.
6. Congestion appears in dense scenarios.
7. Deadlock recovery can be observed.

The exported logs provide quantitative evidence:

1. Number of completed tasks.
2. Throughput.
3. Completion time.
4. Collision count.
5. Invalid moves prevented.
6. Deadlocks detected.
7. Agent utilization.
8. Utility.
9. Social welfare.

A strong validation cannot rely only on animation. It must also use logs and metrics. The final system provides both.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

5.24 Limitations of the Simulator

The simulator is intentionally scoped as a software simulation, not a physical warehouse system.

Main limitations:

1. Communication is assumed reliable.
2. Movement execution is deterministic.
3. Agents do not suffer mechanical failure.
4. Sensors are not noisy.
5. Real-time network delay is not modeled.
6. Physical acceleration and robot dynamics are abstracted away.
7. The warehouse layout is simplified as a grid.
8. Utility is calculated from logged metrics rather than being the only internal optimization objective.

These limitations are acceptable for the project scope because the assignment focuses on MAS strategy design, simulation, visualization, and validation. Adding real robotics constraints would significantly expand the project beyond the intended scope.

5.25 Chapter Summary

This chapter described the simulator and visualization platform used by the project.

The simulator provides a deterministic discrete-time environment for testing multi-agent warehouse coordination. It supports multiple scenarios, two strategy modes, GUI rendering, command-line execution, structured logs, and reproducible metrics.

The Pygame dashboard visualizes agents, tasks, paths, reservations, congestion, conflicts, deadlocks, and live metrics. The exported JSON/CSV files provide quantitative evidence for the validation report.

The simulator is therefore not just a display tool. It is the experimental platform used to prove that the advanced multi-agent strategy works, that it improves over the naive baseline, and that the system satisfies the main requirements defined in the SRS and SDD.

Chapter 6: Results & Validation Analysis

6.1 Overview

This chapter presents the empirical validation of the Swarm Logistics warehouse automation simulator.

The purpose of validation is to prove that the implemented multi-agent strategy works under realistic simulation conditions, not only that the code runs. The evaluation focuses on task completion, throughput, safety, collision prevention, deadlock handling, utility, social welfare, and behavior under stress.

This chapter is especially important because the final report guidelines require empirical evidence of system success, including utility and social welfare charts compared to baseline strategies.

The validation was performed using two levels of testing:

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

1. Direct strategy comparison

The naive baseline strategy was compared against the advanced engineered strategy on the baseline warehouse scenario.

2. Scenario-based stress validation

The advanced strategy was tested across baseline, high-load, high-density, deadlock, dynamic, scalability, and long high-load scenarios.

The main conclusion is clear: the advanced strategy is safer, more productive, and more socially beneficial than the naive baseline. It also remains collision-free across all advanced validation scenarios.

6.2 Validation Objectives

The validation process answers the following questions:

1. Does the system complete pickup-and-delivery tasks successfully?
2. Does the auction mechanism assign each task to one agent only?
3. Does the advanced strategy outperform the naive baseline?
4. Does Cooperative A* with reservations prevent collisions?
5. Does the system prevent edge-swap conflicts?
6. Does the system detect and recover from deadlocks?
7. Does the system remain stable under high task load?
8. Does the system remain stable under high agent density?
9. Does throughput improve when more agents are added?
10. Do utility and social welfare support the claimed improvement?

The validation report previously tested the system through simulation, metric collection, data analysis, and stress testing. It specifically evaluated whether agents complete tasks, whether auction assignment is deterministic, whether Cooperative A* prevents collisions, whether the system handles high load and high density, and whether utility/social welfare are acceptable.

6.3 Evaluation Environment

The simulator was evaluated using a discrete-time Python simulation.

The main implementation configuration was:

Parameter	Value
Language	Python 3.x
Visualization	Pygame
Simulation type	Discrete timestep simulation
Main run script	run_simulation.py
Scenario runner	run_all_scenarios.py
Strategy comparison runner	run_strategy_comparison.py
Output formats	JSON and CSV

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Default grid rows	16
Default grid columns	24
Default baseline agents	6
Default baseline task generation rate	0.18
Default random seed	7
Agent movement	1 grid cell per timestep
Reservation horizon	18 timesteps
Baseline run length	300 timesteps

Table 25: Evaluation environment and default simulation parameters.

The simulator exports summary files, event logs, agent decision traces, and timestep snapshots. These files make the validation reproducible and allow the reported results to be inspected after the run.

6.4 Evaluation Metrics

The system was evaluated using both system-level and agent-level metrics.

6.4.1 System-Level Metrics

Metric	Meaning
tasks_created	Number of generated pickup/drop-off tasks.
tasks_completed	Number of completed tasks.
throughput	Completed tasks divided by simulation timesteps.
avg_completion_time	Average time from task creation to completion.
collision_events	Actual collisions after movement execution.
invalid_moves_prevented	Unsafe moves blocked before execution.
deadlocks_detected	Persistent blocking cycles detected.
agent_utilization	Fraction of agent time spent active.
social_welfare	Sum of agent utilities.
average_utility	Social welfare divided by number of agents.

Table 26: System-level metrics used for validation analysis.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Throughput is defined as:

$$\text{throughput} = \text{completed_tasks} / \text{total_time}$$

For example:

$$\text{throughput} = 42 / 300 = 0.14 \text{ tasks per timestep}$$

This means the system completed approximately 14 tasks per 100 timesteps. The validation report uses this same interpretation of throughput as completed tasks per simulation timestep.

6.4.2 Utility Function

Utility was computed from exported agent metrics.

For each agent i :

$$\begin{aligned} U_i = & 10 \times \text{completed_tasks}_i \\ & - 0.05 \times \text{waiting_time}_i \\ & - 0.10 \times \text{replanning_count}_i \\ & - 0.05 \times \text{conflicts_involved}_i \\ & - 1.00 \times \text{deadlocks_involved}_i \end{aligned}$$

Interpretation:

Term	Meaning
completed_tasks_i	Positive reward for productive task completion.
waiting_time_i	Penalty for lost time due to waiting.
replanning_count_i	Penalty for planning instability.
conflicts_involved_i	Penalty for local coordination pressure.
deadlocks_involved_i	Larger penalty for severe coordination failure.

Table 27: Utility-function terms used to compute agent performance.

The reward for task completion is intentionally dominant. Waiting, replanning, and conflicts are penalized because they represent coordination overhead. Deadlock involvement receives a larger penalty because it represents a stronger coordination failure. The validation report uses this same utility model and explains that task completion should dominate the score while coordination costs still matter.

6.4.3 Social Welfare

Social welfare is defined as the sum of all individual agent utilities:

$$\text{Social Welfare} = \sum U_i$$

Average utility is:

$$\text{Average Utility} = \text{Social Welfare} / \text{Number of Agents}$$

Social welfare measures total system benefit. Average utility makes scenarios with different numbers of agents easier to compare.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

6.5 Strategy Comparison: Naive Baseline vs. Advanced Strategy

The most important strategic validation is the direct comparison between the naive baseline and the advanced engineered strategy.

The naive baseline uses nearest-agent task assignment and standard A*. It does not use auction bidding, congestion-aware bidding, Cooperative A*, time-space reservations, edge-swap prevention, or deadlock recovery.

The advanced strategy uses deterministic auction bidding, Cooperative A*, reservation-table collision prevention, edge-swap prevention, congestion-aware bidding, replanning, and deadlock detection/recovery.

Both strategies were tested on the baseline warehouse scenario for 300 timesteps.

Strategy	Completed Tasks	Throughput	Avg Completion Time	Collision Events	Invalid Moves Prevented	Utilization	Social Welfare	Avg Utility
Naive baseline	12	0.040	40.83	1	1194	0.263	-57.80	-9.63
Advanced strategy	42	0.140	44.10	0	58	0.838	387.35	64.56

Table 28: Naive baseline versus advanced strategy validation results.

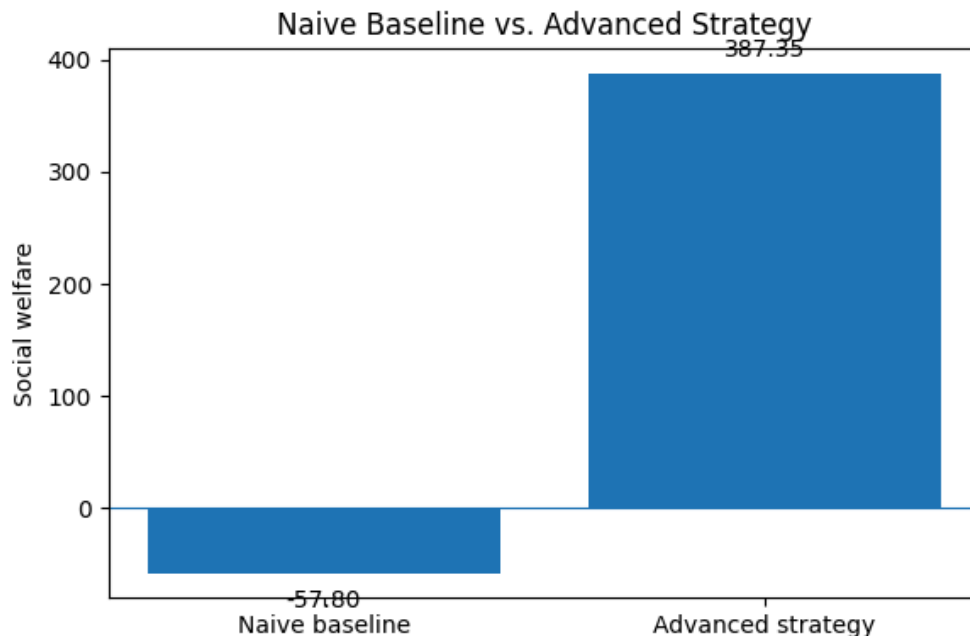


Figure 3: Naive baseline vs. advanced strategy comparison.

The advanced strategy completed 42 tasks compared with only 12 tasks for the naive baseline. This is a 3.5× improvement in completed tasks. Throughput improved from 0.040 tasks per timestep to 0.140 tasks per timestep.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The safety difference is even more important. The naive baseline produced 1 actual collision event, while the advanced strategy produced 0 actual collision events. This shows that standard A* with current occupancy checks is not sufficient for safe multi-agent coordination. The naive baseline also produced 1194 invalid move preventions, compared with only 58 for the advanced strategy. This means the naive agents repeatedly attempted unsafe or blocked movements because they did not coordinate future movement. The advanced strategy avoided most of these failed attempts through Cooperative A*, reservations, and edge-swap prevention. The social welfare result confirms the same conclusion. The naive baseline produced negative social welfare, -57.80, because low task completion and high coordination costs outweighed productive work. The advanced strategy produced positive social welfare, 387.35, and much higher average utility per agent. The validation report's direct comparison shows the same numeric result and concludes that the engineered strategy is more productive, safer, and more socially beneficial than the naive baseline.

6.6 Advanced Strategy Scenario Results

After proving that the advanced strategy outperforms the naive baseline, the advanced strategy was tested across multiple scenarios.

Scenario	Agents	Created	Completed	Throughput	Avg completion	Collisions	Invalid moves prevented	Deadlocks	Utilization	Social welfare	Avg utility
baseline	6	58	42	0.140	44.10	0	58	4	0.838	387.35	64.56
high_load	8	128	49	0.163	85.86	0	194	16	0.802	389.80	48.73
high_density	14	73	57	0.190	47.61	0	865	16	0.457	273.75	19.55
deadlock	4	99	42	0.140	90.69	0	36	8	0.912	382.90	95.73
dynamic	8	81	56	0.187	59.12	0	75	9	0.864	506.70	63.34
scalability	12	96	77	0.257	41.10	0	308	22	0.738	620.60	51.72

Table 29: Advanced-strategy validation results across all tested scenarios.

The key safety result is:
collision_events = 0 in all advanced scenarios

This is the strongest validation outcome. It means the reservation table, conflict detection, edge-swap checks, and final movement safety gate prevented actual collisions even under high load, high density, and deadlock-prone conditions. The validation report records the same scenario table and explicitly notes that all tested advanced scenarios produced zero actual collision events.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

6.7 Throughput Analysis

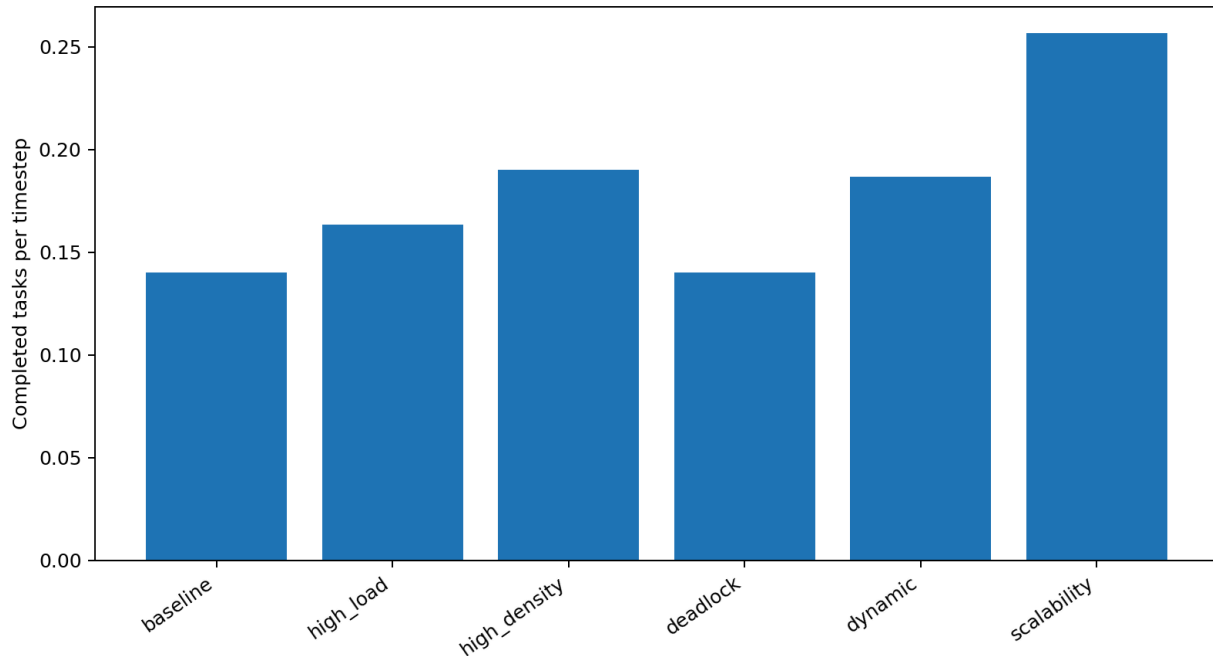


Figure 4: Throughput by scenario.

Throughput measures completed tasks per timestep.
The results show:

Scenario	Throughput
baseline	0.140
high_load	0.163
high_density	0.190
deadlock	0.140
dynamic	0.187
scalability	0.257

Table 30: Throughput comparison across validation scenarios.

The scalability scenario achieved the highest throughput, 0.257 tasks per timestep. This means it completed approximately 25.7 tasks per 100 timesteps.

The throughput increase in scalability is expected because the system has more agents and enough space to let them work in parallel. The scalability scenario completed 77 tasks, the highest task completion count among the 300-step scenario suite.

High-density also improved throughput compared with baseline, but the interpretation is different. High-density completed more tasks than baseline, but it also created much higher

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

congestion pressure. Therefore, high throughput alone does not prove that the system is healthier. It must be interpreted together with completion time, invalid moves, deadlocks, and average utility.

The validation report makes this same point: throughput can increase under load because agents spend less time idle, but higher throughput does not automatically mean better system health.

6.8 Average Completion Time Analysis

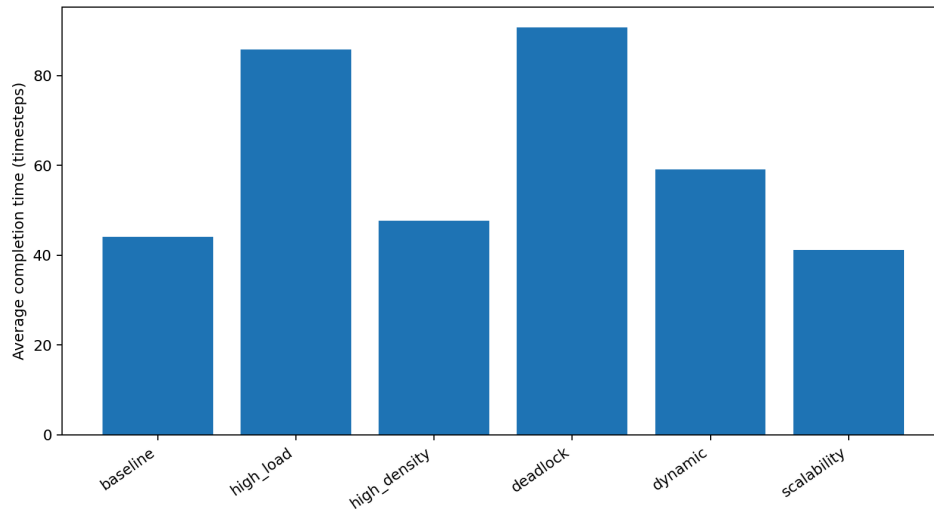


Figure 5: Average completion time by scenario.

Average completion time measures how long tasks take from creation to completion.

Scenario	Avg Completion Time
baseline	44.10
high_load	85.86
high_density	47.61
deadlock	90.69
dynamic	59.12
scalability	41.10

Table 31: Average task completion time across validation scenarios.

The deadlock scenario had the highest average completion time, 90.69 timesteps. This is expected because the scenario is designed around narrow corridors and blocking conditions. The high-load scenario also had high average completion time, 85.86 timesteps. This is expected because tasks are generated faster, so queueing and route contention increase.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The scalability scenario achieved the best average completion time, 41.10 timesteps. This suggests that adding agents can improve completion speed when the grid still has enough space to support parallel movement.

The conclusion is that the advanced strategy benefits from additional agents up to a point. If the environment becomes too dense, coordination overhead begins to reduce individual efficiency.

6.9 Social Welfare Analysis

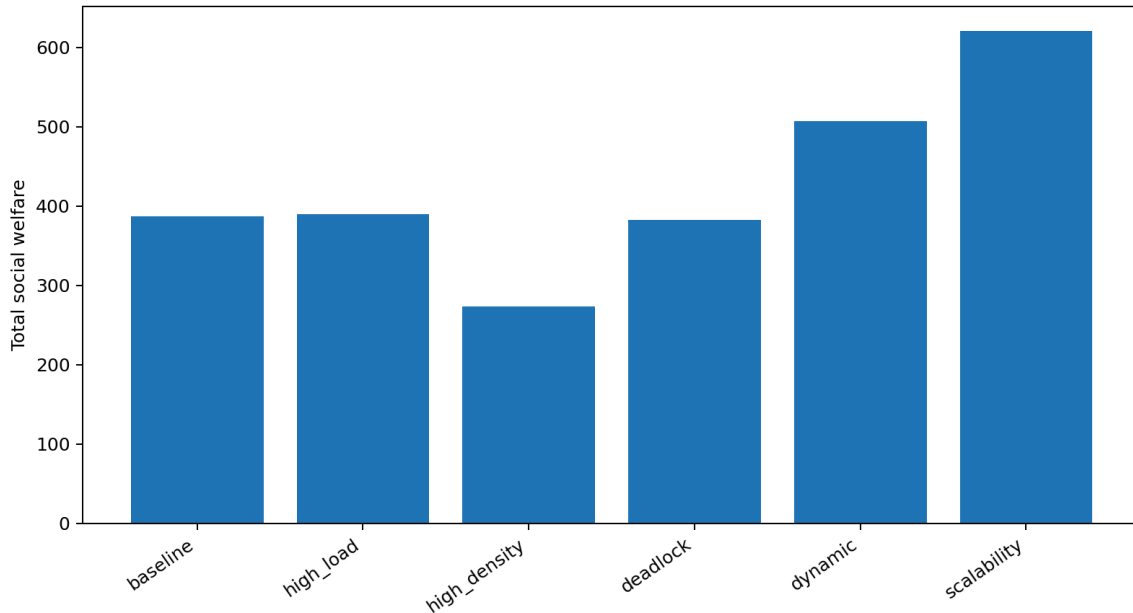


Figure 6: Social welfare by scenario.

Social welfare measures total system benefit after task-completion rewards and coordination penalties.

Scenario	Social Welfare
baseline	387.35
high_load	389.80
high_density	273.75
deadlock	382.90
dynamic	506.70
scalability	620.60

Table 32: Social welfare values across validation scenarios.

The scalability scenario achieved the highest total social welfare, 620.60. This is because it completed the highest number of tasks and maintained zero collisions.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The dynamic scenario achieved the second-highest social welfare, 506.70, and is one of the most balanced scenarios. It improved throughput compared with baseline without creating the severe utility drop seen in the high-density scenario.

The high-density scenario had the lowest social welfare among the advanced strategy scenarios, 273.75. This confirms that adding too many agents into the same grid can reduce system quality even when throughput increases.

The validation report's social welfare analysis also identifies scalability as the highest social-welfare scenario and dynamic as one of the most balanced scenarios.

6.10 Average Utility per Agent Analysis

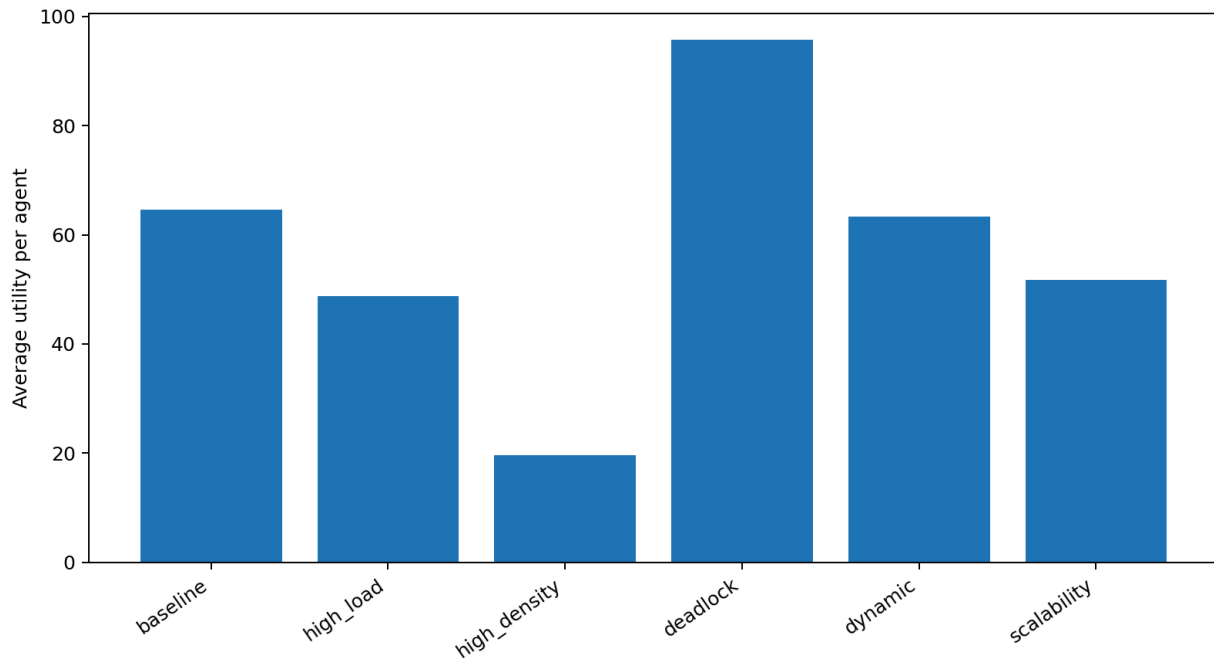


Figure 7: Average utility per agent by scenario.

Average utility makes scenarios with different agent counts easier to compare.

Scenario	Avg Utility
baseline	64.56
high_load	48.73
high_density	19.55
deadlock	95.73
dynamic	63.34
scalability	51.72

Table 33: Average utility per agent across validation scenarios.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The high-density scenario had the lowest average utility per agent, 19.55. This is the clearest evidence that too many agents in the same warehouse grid reduce individual efficiency. The deadlock scenario had the highest average utility per agent, 95.73. This does not mean it is the best operational scenario. It used only four agents, and those agents still completed 42 tasks. However, its average completion time was high, meaning tasks took longer to finish. Therefore, deadlock should not be treated as the best overall scenario. The dynamic scenario had average utility close to baseline while achieving higher throughput and higher social welfare. This makes it one of the most balanced scenario configurations.

6.11 Invalid Move Prevention Analysis

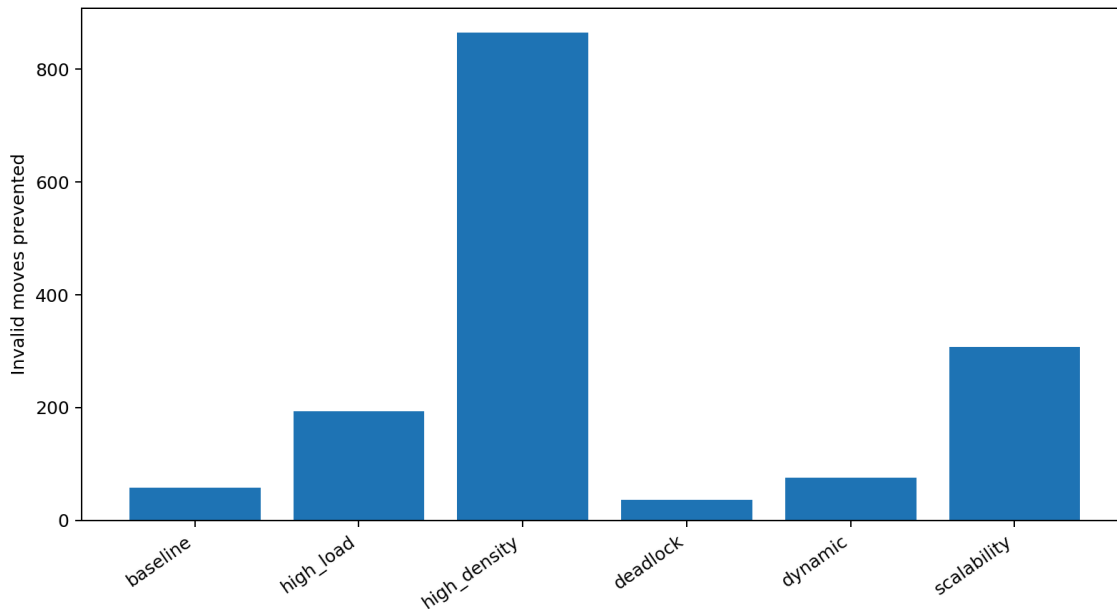


Figure 8: Invalid moves prevented by scenario.

Invalid moves prevented are not collisions. They are unsafe actions blocked before execution.

Scenario	Invalid Moves Prevented
baseline	58
high_load	194
high_density	865
deadlock	36
dynamic	75
scalability	308

Table 34: Invalid moves prevented across validation scenarios.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The high-density scenario produced 865 invalid move preventions. This is far higher than all other scenarios. That result is expected because 14 agents were operating in the same grid and competing for shared corridors and stations.

The important result is that even with 865 invalid move preventions, the high-density scenario still produced 0 actual collisions. This proves that the system was not safe by luck. It was safe because the conflict-prevention logic actively blocked unsafe movement before execution.

The validation report explicitly highlights the high-density scenario as strong evidence because it had 865 invalid moves prevented but still zero actual collisions.

6.12 Deadlock Analysis

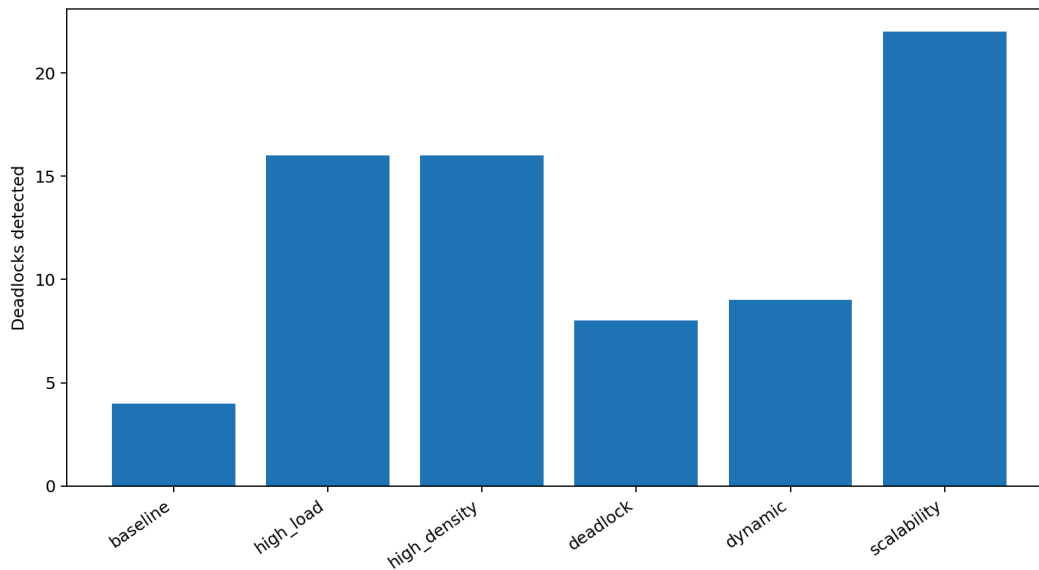


Figure 9: Deadlocks detected by scenario.

Deadlocks were detected in multiple scenarios.

Scenario	Deadlocks Detected
baseline	4
high_load	16
high_density	16
deadlock	8
dynamic	9
scalability	22

Table 35: Deadlocks detected across validation scenarios.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The scalability scenario had the highest number of detected deadlocks, 22. This is expected because more agents increase the probability of cyclic blocking.

High-load and high-density also produced many deadlocks, 16 each. This is expected because both scenarios increase contention: high-load increases task pressure, and high-density increases physical congestion.

The important point is not that deadlocks never occurred. In dense multi-agent systems, some deadlocks are expected. The important result is that the system detected and recovered from them while maintaining zero actual collisions.

The validation report makes the same argument: deadlocks appeared because agents shared narrow corridors, pickup/drop-off regions, and reservation windows, but detection and recovery acted as resilience mechanisms rather than failure points.

6.13 Stress Testing

Stress testing checks whether the system remains functional when conditions become difficult.

The project included four main stress tests:

1. High-load scenario.
2. High-density scenario.
3. Deadlock scenario.
4. Long high-load run for 6000 timesteps.

6.13.1 High-Load Scenario

The high-load scenario increased task generation pressure and used 8 agents.

Metric	Value
Tasks created	128
Tasks completed	49
Throughput	0.163
Avg completion time	85.86
Collision events	0
Invalid moves prevented	194
Deadlocks detected	16
Agent utilization	0.802
Social welfare	389.80
Average utility	48.73

Table 36: High-load stress-test results.

The high-load scenario completed more work per timestep than baseline. Throughput increased from 0.140 to 0.163.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

However, average completion time also increased from 44.10 to 85.86. This means the system became more saturated. More tasks were available, so agents were productive, but the queue and route contention increased.

This is the expected high-load behavior: throughput increases until the system approaches saturation, while waiting and completion time also rise.

6.13.2 High-Density Scenario

The high-density scenario used 14 agents in the same warehouse grid.

Metric	Value
Tasks created	73
Tasks completed	57
Throughput	0.190
Avg completion time	47.61
Collision events	0
Invalid moves prevented	865
Deadlocks detected	16
Agent utilization	0.457
Social welfare	273.75
Average utility	19.55

Table 37: High-density stress-test results.

This was the strongest congestion test.

The system completed 57 tasks, more than baseline. However, agent utilization dropped to 0.457, and average utility fell to 19.55. This shows that many agents were present, but they were not all being used efficiently.

The high invalid-move count, 865, shows intense coordination pressure. The result is still positive from a safety perspective because collision events remained zero.

Conclusion: high density increases total work, but it reduces individual efficiency because agents block each other more often.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

6.13.3 Deadlock Scenario

The deadlock scenario used a constrained layout designed to provoke blocking.

Metric	Value
Tasks created	99
Tasks completed	42
Throughput	0.140
Avg completion time	90.69
Collision events	0
Deadlocks detected	8
Agent utilization	0.912
Social welfare	382.90
Average utility	95.73

Table 38: Deadlock stress-test results.

The system maintained zero collisions despite the difficult layout. The average completion time was high, which confirms that the scenario was hard.

The key result is that the simulation continued. Deadlock recovery allowed agents to resume movement rather than halt permanently.

The validation report states the same conclusion: the deadlock scenario maintained zero collisions, and deadlock recovery allowed the simulation to continue despite high average completion time.

6.13.4 Long High-Load Run: 6000 Timesteps

A long high-load run was executed for 6000 timesteps.

Metric	Value
Total time	6000
Tasks created	2826
Tasks completed	1172
Throughput	0.195
Avg completion time	1777.79
Collision events	0

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Invalid moves prevented	2788
Deadlocks detected	321
Agent utilization	0.878
Social welfare	10002.55
Average utility	1250.32

Table 39: Long high-load stress-test results over 6000 timesteps.

This is the strongest long-run stability test.

The system completed 1172 tasks and maintained zero actual collisions over 6000 timesteps.

That is a strong safety result.

However, average completion time became very high, 1777.79 timesteps. This means task generation exceeded the system's long-term clearing capacity. The agents remained safe and productive, but the task queue grew under sustained pressure.

This is not a failure of collision avoidance. It is a saturation result. Under sustained high demand, the system needs more warehouse capacity, lower task rate, more pickup/drop-off diversity, larger grid layout, or additional congestion-aware throttling.

The validation report reaches the same interpretation: the long run confirms long-run safety, but the very high completion time shows that task generation exceeded long-term clearing capacity.

6.14 Requirement Validation

The validation results support the main SRS and SDD requirements.

Requirement	Evidence
Dynamic task generation	All scenarios generated tasks during runtime.
Task lifecycle tracking	Tasks were created, assigned, picked up, and completed.
Single task ownership	Deterministic assignment gave each task one owner.
Collision avoidance	Advanced scenarios had 0 actual collisions.
Movement validity	Invalid moves were blocked before execution.
Edge-swap prevention	Edge conflicts were detected and prevented.
Deadlock detection	Deadlocks were detected in stress scenarios.
Deadlock recovery	Simulation continued after deadlock events.
Congestion handling	High-density and high-load behavior showed congestion pressure and safe adaptation.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Path planning	Agents completed tasks under baseline, dynamic, scalability, and stress scenarios.
Dynamic replanning	Invalid moves, waiting, and deadlock recovery triggered replanning behavior.
Logging and observability	JSON/CSV outputs recorded summaries, events, decisions, and snapshots.
Utility/social welfare	Utility and social welfare were computed and compared across scenarios and strategies.

Table 40: Requirement validation summary for the final implementation.

The validation report also maps results to SRS/SDD requirements, including dynamic task generation, auction synchronization, collision avoidance, cooperative path planning, deadlock handling, and metrics/logging evidence.

6.15 Main Findings

The validation supports five main findings.

6.15.1 The Advanced Strategy Outperforms the Naive Baseline

The advanced strategy completed 42 tasks compared with 12 for the naive baseline. It also reduced invalid moves from 1194 to 58 and improved social welfare from -57.80 to 387.35. This confirms that the advanced coordination mechanisms are necessary. The improvement comes from auction bidding, Cooperative A*, time-space reservations, edge-swap prevention, and deadlock recovery.

6.15.2 Collision Avoidance Works

The advanced strategy produced zero actual collision events in all tested scenarios. This is the strongest safety result in the project. It validates the reservation table, movement conflict checks, edge-swap prevention, and final safety gate.

6.15.3 Higher Throughput Does Not Always Mean Better System Health

High-density achieved higher throughput than baseline, but its average utility per agent was much lower.

This means throughput must be interpreted with completion time, invalid moves, deadlocks, utilization, and utility. A system can complete more total tasks while becoming less efficient per agent.

6.15.4 More Agents Help Only Until Congestion Costs Dominate

The scalability scenario achieved the highest throughput and social welfare. However, high-density showed that adding too many agents to the same grid creates coordination overhead.

This is a classic MAS tradeoff: parallelism improves productivity until shared-resource contention becomes the dominant cost.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

6.15.5 Deadlocks Are Manageable, Not Impossible

Deadlocks occurred in stress scenarios, but they did not break the simulation. The system detected persistent blocking and recovered without collisions.

This is realistic. In dense multi-agent environments, preventing every possible blocking pattern is difficult. A practical strategy must detect and recover from deadlocks rather than pretend they will never occur.

6.16 Threats to Validity

The validation is strong for a simulation project, but it has limitations.

1. Simulation-only validation
The system is a discrete-time Python simulation, not a physical robot deployment.
2. Deterministic movement
Real robots would have actuation delays, localization errors, and mechanical uncertainty.
3. Reliable communication assumption
The simulation assumes communication is reliable and timely.
4. Fixed warehouse layouts
The tested layouts are deterministic and limited. More layouts would improve generality.
5. Utility is derived from logs
Utility and social welfare are computed from exported metrics, not directly optimized as the only internal decision function.
6. Naive baseline comparison scope
The naive baseline comparison was run on the baseline scenario. More comparisons across stress scenarios would strengthen statistical evidence further.
7. Limited statistical repetition
The reported results are scenario-based deterministic runs. Running each scenario across many random seeds would allow confidence intervals and variance analysis.

These limitations do not invalidate the results. They define the scope of the evidence. For the project requirements, the validation demonstrates that the advanced strategy works, improves over baseline, and remains safe under stress.

6.17 Final Validation Conclusion

The validation results support the final claim of the project:

The advanced multi-agent coordination strategy successfully improves task completion, safety, and social welfare compared with a naive baseline while maintaining collision-free operation across multiple stress scenarios.

The most important evidence is:

1. Advanced strategy completed 42 tasks vs. 12 for the naive baseline.
2. Advanced strategy achieved 0 collisions vs. 1 collision for the naive baseline.
3. Advanced strategy reduced invalid move attempts from 1194 to 58 in the baseline comparison.
4. Advanced strategy improved social welfare from -57.80 to 387.35.
5. All advanced scenarios produced 0 actual collision events.
6. Scalability achieved the highest throughput: 0.257 tasks per timestep.
7. Long high-load testing completed 1172 tasks over 6000 timesteps with 0 collisions.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

8. Deadlocks were detected and handled without halting the simulation. Overall, the implemented MAS satisfies the main project goals. It demonstrates decentralized-style task allocation, cooperative path planning, conflict prevention, deadlock recovery, congestion-aware behavior, real-time visualization, and measurable validation through utility and social welfare.

Chapter 7: Social, Ethical & Future Work

7.1 Overview

This chapter discusses the broader implications, limitations, and future directions of the Swarm Logistics warehouse automation system.

Although the project is implemented as a simulation, it represents a realistic class of autonomous systems: multiple robots operating in a shared environment while making local decisions that affect global performance. Therefore, the system must be evaluated not only by throughput, but also by safety, transparency, accountability, fairness, and practical deployment limitations.

The final implementation addresses these concerns at the simulation level through collision prevention, deterministic decision rules, deadlock recovery, structured logs, reproducible scenarios, and measurable utility/social welfare analysis.

7.2 Social Impact

Warehouse automation can improve logistics efficiency by increasing order throughput, reducing repetitive physical labor, and improving operational consistency. In a real warehouse, autonomous robots can reduce the need for humans to perform repetitive movement tasks and can support faster fulfillment under high demand.

However, automation also has social risks. It may reduce demand for some manual warehouse roles while increasing demand for technical roles such as robot supervision, maintenance, system monitoring, and exception handling. A responsible automation system should therefore be treated as supervised automation, not as an uncontrolled replacement for human judgment. The main social tradeoff is clear: automation can improve productivity, but it must remain safe, explainable, and controllable.

7.3 Ethical Considerations

7.3.1 Safety

Safety is the most important ethical requirement.

In the simulation, safety means:

1. No two agents occupy the same cell at the same timestep.
2. Agents do not move into obstacles.
3. Agents do not move outside the warehouse boundary.
4. Agents do not perform edge swaps.
5. Unsafe moves are blocked before execution.
6. Deadlocks are detected and recovered from.

The advanced strategy achieved zero actual collisions across all tested scenarios. This is the strongest safety evidence in the project.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

For real deployment, additional safety mechanisms would be required, including emergency stops, speed limits, human detection, sensor redundancy, certified safety zones, and fail-safe behavior under communication or sensor failure. Therefore, this project demonstrates algorithmic safety in simulation, not physical safety certification.

7.3.2 Transparency and Explainability

Autonomous warehouse agents should not behave like black boxes. If an agent waits, replans, loses priority, or changes route, the system should make the reason visible.

The implementation supports explainability through:

1. Event logs.
2. Agent decision logs.
3. Task assignment records.
4. Path and reservation visualization.
5. Invalid-move prevention records.
6. Deadlock detection and recovery logs.
7. Scenario summaries and exported metrics.

This makes the system auditable. If performance drops or an unexpected behavior appears, the logs can be inspected to identify whether the cause was task assignment, path planning, congestion, blocked movement, or deadlock recovery.

7.3.3 Accountability

A real autonomous warehouse system must allow failures to be traced. The final simulator supports accountability through deterministic execution and structured logging.

Deterministic tie-breaking is especially important. If two agents submit equal bids, the lower agent ID wins. If priority rules tie, deterministic rules resolve the conflict. This avoids random behavior and makes repeated validation possible.

This improves accountability because the same input configuration produces the same result.

7.3.4 Fairness Between Agents

Fairness means the system should not repeatedly overload one agent or repeatedly deprioritize another without reason.

The auction mechanism helps fairness by considering workload, distance, congestion, and reservation delay. The exported metrics also allow fairness inspection through:

1. Tasks completed per agent.
2. Active time.
3. Idle time.
4. Waiting time.
5. Replanning count.
6. Conflicts involved.
7. Deadlocks involved.

The current system does not implement a formal fairness optimizer, but it provides the data needed to detect unfair task distribution and improve it in future versions.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

7.3.5 Productivity vs. Safety

A common automation risk is optimizing productivity while treating safety as secondary. This project avoids that mistake.

Throughput is important, but collision avoidance is non-negotiable. A scenario with high throughput but nonzero collisions should be considered unsafe. This is why the report separates actual collisions from invalid moves prevented.

An invalid move prevented means the system blocked unsafe behavior. An actual collision means the safety mechanism failed. The advanced strategy's zero-collision result is therefore more important than its throughput result.

7.4 Limitations

The final implementation is strong as a simulation project, but it has clear limitations.

1. Simulation-only scope
The system is not deployed on real robots and does not model sensors, motors, localization errors, robot dynamics, emergency stops, or human movement.
2. Simplified grid model
The warehouse is represented as a discrete grid. Real robots operate in continuous space and must handle turning radius, acceleration, braking distance, and physical size.
3. Reliable communication assumption
The simulator assumes communication is reliable and has negligible delay. A real system would need to handle packet loss, latency, message ordering, and disconnections.
4. Deterministic movement assumption
The simulator assumes valid movement always succeeds. Real robots may experience wheel slip, localization drift, mechanical failure, or unexpected obstacles.
5. Limited baseline scope
The naive baseline was compared numerically against the advanced strategy on the baseline scenario. Future work should extend the comparison to all stress scenarios.
6. Limited statistical repetition
The validation uses deterministic scenario runs. A stronger scientific evaluation would run many seeds and report mean, standard deviation, and confidence intervals.
7. Utility is derived from logs
Utility and social welfare are computed from exported metrics. They are used mainly for evaluation, not as the only direct internal optimization objective.

These limitations define the scope of the evidence. They do not invalidate the project, but they clarify what the simulation proves and what it does not prove.

7.5 Future Work

Several extensions could improve realism, robustness, and scientific strength.

7.5.1 Multi-Seed Statistical Evaluation

Each scenario should be executed across many random seeds. This would allow the report to include averages, standard deviations, confidence intervals, and failure frequency.

This would answer whether the zero-collision result and throughput trends are consistent across different task-generation sequences.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

7.5.2 Baseline Comparison Across All Scenarios

The naive baseline should be tested not only on the baseline scenario, but also on high-load, high-density, deadlock, dynamic, and scalability scenarios.

This would likely show that the naive strategy degrades more severely under congestion and would make the advanced-strategy comparison stronger.

7.5.3 Energy-Aware Coordination

Future versions could add energy cost to the utility function.

Energy cost could depend on:

1. Distance traveled.
2. Waiting time.
3. Number of turns.
4. Carrying an item.
5. Replanning frequency.
6. Congestion-related detours.

This would make the simulation closer to real warehouse operations, where battery life and charging schedules matter.

7.5.4 More Realistic Warehouse Layouts

The current layouts are sufficient for validation, but future work could include:

1. Multiple aisles.
2. One-way lanes.
3. Loading zones.
4. Charging stations.
5. Human-only areas.
6. Dynamic blocked areas.
7. Larger warehouse sections.

This would improve generality and make the simulation more realistic.

7.5.5 Dynamic Obstacles and Human Workers

Real warehouses include humans, forklifts, temporary obstacles, and blocked aisles. Future work could add dynamic obstacles that appear and move during runtime.

This would test whether agents can replan safely under more realistic uncertainty.

7.5.6 Communication Failure Modeling

Future work should model communication delays and failures.

Possible additions include:

1. Message delay.
2. Message loss.
3. Limited communication radius.
4. Out-of-order messages.
5. Agent disconnection.
6. Safe fallback behavior when reservations cannot be confirmed.

This would test whether decentralized coordination remains stable under imperfect communication.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

7.5.7 Learning-Based Adaptation

The OnePager mentioned reinforcement learning as an optional extension. This remains a useful direction, especially for learning better auction weights, congestion penalties, replanning thresholds, or task assignment policies.

However, learning should not replace hard safety constraints. Collision prevention, edge-swap prevention, and emergency-stop behavior should remain rule-based constraints. Learning should optimize efficiency only inside a safe action space.

7.5.8 Replay and Debugging Tools

The GUI could be extended with replay tools:

1. Save full simulation replays.
2. Move backward and forward through timesteps.
3. Click an agent to inspect its decision history.
4. Click a task to inspect its lifecycle.
5. Visualize wait-for graphs during deadlocks.
6. Compare naive and advanced strategies side by side.

This would improve debugging, demonstration, and grading clarity.

7.6 Lessons Learned

The project demonstrates several important MAS engineering lessons.

1. Local decisions are not enough.
Nearest-agent assignment looks reasonable but performs poorly without coordination.
2. Future-aware planning matters.
Cooperative A* and reservations prevent conflicts before they happen.
3. Safety must be enforced before execution.
Unsafe moves must be blocked, not merely detected after collision.
4. More agents are not always better.
High-density scenarios show that coordination cost can reduce average utility.
5. Deadlocks should be expected.
Dense shared environments can create cyclic blocking. A robust system detects and recovers instead of assuming deadlocks will never happen.
6. Visualization and logs are part of validation.
The GUI explains behavior visually, while exported JSON/CSV files prove results numerically.

7.7 Final Project Conclusion

Swarm Logistics & Warehouse Automation successfully demonstrates a complete multi-agent warehouse coordination system at the simulation level.

The final system includes dynamic task generation, autonomous robot agents, deterministic auction-based task allocation, a naive baseline strategy, Standard A*, Cooperative A*, time-space reservations, vertex and edge-swap conflict prevention, priority-based conflict resolution, dynamic replanning, deadlock recovery, Pygame visualization, JSON/CSV logging, utility analysis, social welfare analysis, and scenario-based validation.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The validation results show that the advanced strategy significantly outperforms the naive baseline. It completes more tasks, eliminates collisions, reduces invalid movement attempts, and produces much higher social welfare.

The most important final result is:

`collision_events = 0`

across all advanced validation scenarios.

This means the project achieved its most important safety objective at the simulation level. The project also shows realistic tradeoffs. Higher task load can increase throughput but also increase completion time. More agents can improve productivity but may reduce average utility under congestion. Deadlocks can occur, but they can be detected and recovered from. Overall, the final implementation satisfies the project vision: it demonstrates how decentralized-style multi-agent coordination can support safe and efficient warehouse automation through auction-based task allocation, cooperative path planning, conflict prevention, deadlock recovery, visualization, and empirical validation.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

References

1. Swarm Logistics & Warehouse Automation – OnePager. Project selection, environment characterization, action strategies, and success metrics.
2. Swarm Logistics & Warehouse Automation – Software Requirements Specification (SRS). Final environment model, agent specification, functional requirements, simulation requirements, interaction model, utility metrics, and validation scenarios.
3. Swarm Logistics & Warehouse Automation – Software Design Document (SDD). Final architecture, agent reasoning model, communication design, auction protocol, Cooperative A*, reservation table, conflict handling, deadlock detection, implementation design, visualization design, and analytics design.
4. Swarm Logistics & Warehouse Automation – Validation & Testing Report. Strategy comparison, validation scenarios, stress testing, utility/social welfare analysis, and final empirical results.
5. Python Software Foundation. Python 3 Documentation.
6. Pygame Documentation. Pygame library for real-time visualization and event handling.
7. Hart, P. E., Nilsson, N. J., and Raphael, B. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” IEEE Transactions on Systems Science and Cybernetics, 1968.
8. Silver, D. “Cooperative Pathfinding.” Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 2005.
9. Foundation for Intelligent Physical Agents. FIPA ACL Message Structure Specification.

Appendix A: How to Run the Simulator

A.1 Requirements

The simulator requires Python 3.x.

Install the required packages:

```
pip install -r requirements.txt
```

If Pygame is missing, install it manually:

```
pip install pygame
```

The simulation logic can run without the GUI. The graphical dashboard requires Pygame and the `--render` flag.

A.2 Running a Single Advanced Scenario

Run the baseline scenario using the advanced strategy:

```
python run_simulation.py --scenario baseline --strategy advanced --steps 300 --out runs/baseline_advanced
```

Run the baseline scenario with visualization:

```
python run_simulation.py --scenario baseline --strategy advanced --steps 600 --render --out runs/baseline_gui
```

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

A.3 Running the Naive Baseline

Run the naive baseline strategy:

```
python run_simulation.py --scenario baseline --strategy naive --steps 300 --out runs/baseline_naive
```

This is used for direct comparison against the advanced strategy.

A.4 Running the Strategy Comparison

Run the naive-vs-advanced comparison:

```
python run_strategy_comparison.py
```

This generates:

```
runs/strategy_comparison/strategy_comparison_summary.json
```

A.5 Running All Validation Scenarios

Run all predefined advanced-strategy validation scenarios:

```
python run_all_scenarios.py
```

This executes the scenario suite used in the validation analysis.

A.6 Running the Long Stress Test

Run the long high-load stress test:

```
python run_simulation.py --scenario high_load --strategy advanced --steps 6000 --out runs/high_load_6000
```

A.7 GUI Controls

When running with --render, the following controls are available:

Control	Function
SPACE	Pause or resume the simulation.
S	Step once while paused.
R	Reset the simulation.
P	Show or hide planned paths.
V	Show or hide reservations.
H	Show or hide congestion heatmap.
T	Show or hide task links.

Table 41: GUI controls available when running the simulator.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

Appendix B: Scenario Configuration

The simulator includes several predefined validation scenarios.

Scenario	Agents	Task Generation Rate	Purpose
baseline	6	0.18	Normal operating condition.
high_load	8	0.45	Tests frequent task generation and saturation behavior.
high_density	14	0.22	Tests congestion with many agents in the same grid.
deadlock	4	0.35	Tests narrow-corridor blocking and deadlock recovery.
dynamic	8	0.25	Tests dynamic task behavior under moderate pressure.
scalability	12	0.30	Tests larger agent count and throughput scaling.
long high-load	8	0.45	Tests extended stability over 6000 timesteps.

Table 42: Scenario configuration used for validation experiments.
All scenarios use deterministic random seeds for reproducibility.

Appendix C: Validation Evidence

C.1 Strategy Comparison Summary

The direct naive-vs-advanced comparison was run on the baseline scenario for 300 timesteps.

Strategy	Completed Tasks	Throughput	Avg Completion Time	Collision Events	Invalid Moves Prevented	Utilization	Social Welfare	Avg Utility
Naive baseline	12	0.040	40.83	1	1194	0.263	-57.80	-9.63
Advanced strategy	42	0.140	44.10	0	58	0.838	387.35	64.56

Table 43: Strategy comparison summary used as validation evidence.

The advanced strategy completed 3.5 times more tasks than the naive baseline, eliminated actual collisions, reduced invalid movement attempts, and achieved much higher social welfare.

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

C.2 Advanced Scenario Summary

Scenario	Agents	Created	Completed	Throughput	Avg completion	Collisions	Invalid moves prevented	Deadlocks	Utilization	Social welfare	Avg utility
baseline	6	58	42	0.140	44.10	0	58	4	0.838	387.35	64.56
high_load	8	128	49	0.163	85.86	0	194	16	0.802	389.80	48.73
high_density	14	73	57	0.190	47.61	0	865	16	0.457	273.75	19.55
deadlock	4	99	42	0.140	90.69	0	36	8	0.912	382.90	95.73
dynamic	8	81	56	0.187	59.12	0	75	9	0.864	506.70	63.34
scalability	12	96	77	0.257	41.10	0	308	22	0.738	620.60	51.72

Table 44: Advanced scenario summary used as validation evidence.

The most important validation result is that all advanced-strategy scenarios produced zero actual collision events.

C.3 Long High-Load Stress Test

Metric	Value
Total time	6000
Tasks created	2826
Tasks completed	1172
Throughput	0.195
Avg completion time	1777.79
Collision events	0
Invalid moves prevented	2788
Deadlocks detected	321
Agent utilization	0.878
Social welfare	10002.55
Average utility	1250.32

Detailed Final Project Report

Swarm Logistics – Warehouse Automation Simulation

The long stress test confirms that the advanced strategy remains collision-free over extended operation. The high average completion time shows that the high-load setting exceeded the system's long-term clearing capacity, but the system remained safe and productive.

C.4 Exported Validation Files

Each simulation run exports structured evidence files.

File	Purpose
summary.json	Final run metrics and agent-level metrics.
events.json	Full chronological event log.
events.csv	Spreadsheet-readable event log.
agent_decisions.json	Agent decision traces.
snapshots.json	Timestep-level system snapshots.
snapshots.csv	Spreadsheet-readable snapshot data.
strategy_comparison_summary.json	Naive-vs-advanced comparison output.

Table 46: Exported validation files

These files allow the validation claims in the report to be traced back to generated simulation evidence.